TextGrad: Automatic "Differentiation" via Text

Mert Yuksekgonul[1*] MERTY@STANFORD.EDU
Federico Bianchi[1*] FEDE@STANFORD.EDU
Joseph Boen[2*] TBOEN@STANFORD.EDU
Sheng Liu[2*] SHENGL@STANFORD.EDU
Zhi Huang[2*] ZHIHUANG@STANFORD.EDU
Carlos Guestrin[1,3] GUESTRIN@STANFORD.EDU
James Zou[1,2,3] JAMESZ@STANFORD.EDU
[1]DEPARTMENT OF COMPUTER SCIENCE, STANFORD UNIVERSITY
[2]DEPARTMENT OF BIOMEDICAL DATA SCIENCE, STANFORD UNIVERSITY
[3]CHAN ZUCKERBERG BIOHUB
CORRESPONDENCE: MERTY@STANFORD.EDU AND JAMESZ@STANFORD.EDU

REPOSITORY AND TUTORIALS

arXiv:2406.07496v1 [cs.CL] 11 Jun 2024

Abstract

AI is undergoing a paradigm shift, with breakthroughs achieved by systems orchestrating multiple large language models (LLMs) and other complex components. As a result, developing principled and automated optimization methods for compound AI systems is one of the most important new challenges. Neural networks faced a similar challenge in its early days until backpropagation and automatic differentiation transformed the field by making optimization turn-key. Inspired by this, we introduce **TEXTGRAD**, a powerful framework performing automatic "differentiation" via text. **TEXTGRAD** backpropagates textual feedback provided by LLMs to improve individual components of a compound AI system. In our framework, LLMs provide rich, general, natural language suggestions to optimize variables in computation graphs, ranging from code snippets to molecular structures. **TEXTGRAD** follows PyTorch's syntax and abstraction and is flexible and easy-to-use. It works out-of-the-box for a variety of tasks, where the users only provide the objective function without tuning components or prompts of the framework. We showcase **TEXTGRAD**'s effectiveness and generality across a diverse range of applications, from question answering and molecule optimization to radiotherapy treatment planning. Without modifying the framework, **TEXTGRAD** improves the zero-shot accuracy of GPT-4o in Google-Proof Question Answering from 51% to 55%, yields 20% relative performance gain in optimizing LeetCode-Hard coding problem solutions, improves prompts for reasoning, designs new druglike small molecules with desirable *in silico* binding, and designs radiation oncology treatment plans with high specificity. **TEXTGRAD** lays a foundation to accelerate the development of the next-generation of AI systems.

1 Introduction

There is an emerging paradigm shift in how AI systems are built, owing to the breakthroughs of Large Language Models (LLMs) [1–6]. The new generation of AI applications are increasingly compound systems involving multiple sophisticated components, where each component could be an LLM-based agent, a tool such as a simulator, or web search. For instance, a system of LLMs communicating with symbolic solvers can solve olympiad-level math problems [7]; a system of LLMs using search engines and code interpreter tools performs comparably to human competitive programmers [8] and are solving real-world

*Co-first authors.

GitHub issues [9]. However, many of these breakthroughs came from systems that are hand-crafted by experts in the domain of application and are tweaked through heuristics. Therefore, developing principled and automated ways to optimize AI systems is one of the most crucial challenges for building compound systems with LLMs, and is necessary for unlocking the power of AI [10–12].

For the past 15 years, many advances in AI have relied on artificial neural networks and differentiable optimization [7, 13–17]. Different parts of neural networks (e.g., two artificial neurons) communicate through differentiable functions like matrix multiplications [18]. Therefore, using numerical gradients and backpropagation [19], which provide the direction to adjust each parameter to improve a model, has been the natural way to train AI models. Flexible automatic differentiation frameworks implementing backpropagation [20–24] have been indispensible to the development of AI *models*.

To optimize the new generation of AI *systems*, we introduce **TEXTGRAD**, automatic differentiation via text. Here we use differentiation and gradients as a metaphor for textual feedback from LLMs. In this framework, each AI system is transformed into a computation graph, where variables are inputs and outputs of complex (not necessarily differentiable) function calls. The feedback to the variables (dubbed 'textual gradients' [25]) are provided in the form of informative and interpretable natural language criticism to the variables; describing how a variable should be changed to improve the system. The gradients are propagated through arbitrary functions, such as LLM API calls, simulators, or external numerical solvers.

We demonstrate the power of our framework in a diverse set of domains, ranging from question answering benchmarks to radiotherapy treatment plan optimization and molecule generation (Fig. 1). LLMs can provide very rich, legible, and expressive natural language gradients to variables in this wide range of domains, such as proposing modifications to molecules, prompts to other LLMs, and code snippets. Our framework is built on the assumption that the current state-of-the-art LLMs are able to reason about individual components and subtasks of the system that it tries to optimize. We demonstrate the flexibility of **TEXTGRAD** with the following results:

1. **Coding:** In Section 3.1, we optimize solutions to difficult coding problems from LeetCode [26], where we boost the performance of `gpt-4o` and best existing method by 20% relevant performance gain.
2. **Problem Solving:** In Section 3.2, we optimize solutions to complex scientific questions to improve the zero-shot performance of GPT-4o. For instance, in Google-Proof Question Answering [27] benchmark, we improve the zero-shot accuracy from 51% to 55% by refining the solutions at test-time.
3. **Reasoning:** In Section 3.3, we optimize prompts to improve the LLM performance, where we push the performance of GPT-3.5 close to GPT-4 in several reasoning tasks.
4. **Chemistry:** In Section 3.4, we design new small molecules with desirable druglikeness and *in silico* binding affinity to drug targets.
5. **Medicine:** In Section 3.5, we optimize radiation treatment plans for prostate cancer patients to achieve desirable target dosage and reduce side effects.

Our results in a broad set of applications demonstrate the promise of **TEXTGRAD** to automatically optimize compound AI systems via backpropagation of text feedback. To accelerate progress in this direction, we open-source our framework at `https://github.com/zou-group/textgrad`.

2 TEXTGRAD: Optimizing AI systems by backpropagating text feedback

Below we first describe a example to demonstrate what **TEXTGRAD** looks like for a system made of two LLM calls, and then give the more general form for arbitrarily complex systems.

Warmup: system with two LLM calls

Example computation graph. In traditional automatic differentiation, we compute gradients that provides a direction that would improve the variable with respect to a downstream loss with the chain-rule. For a simple analog, let us look at the simple system:

$$\text{Prediction} = \text{LLM}(\textbf{Prompt} + \text{Question}), \tag{1}$$

$$\text{Evaluation} = \text{LLM}(\text{Evaluation Instruction} + \text{Prediction}), \tag{2}$$

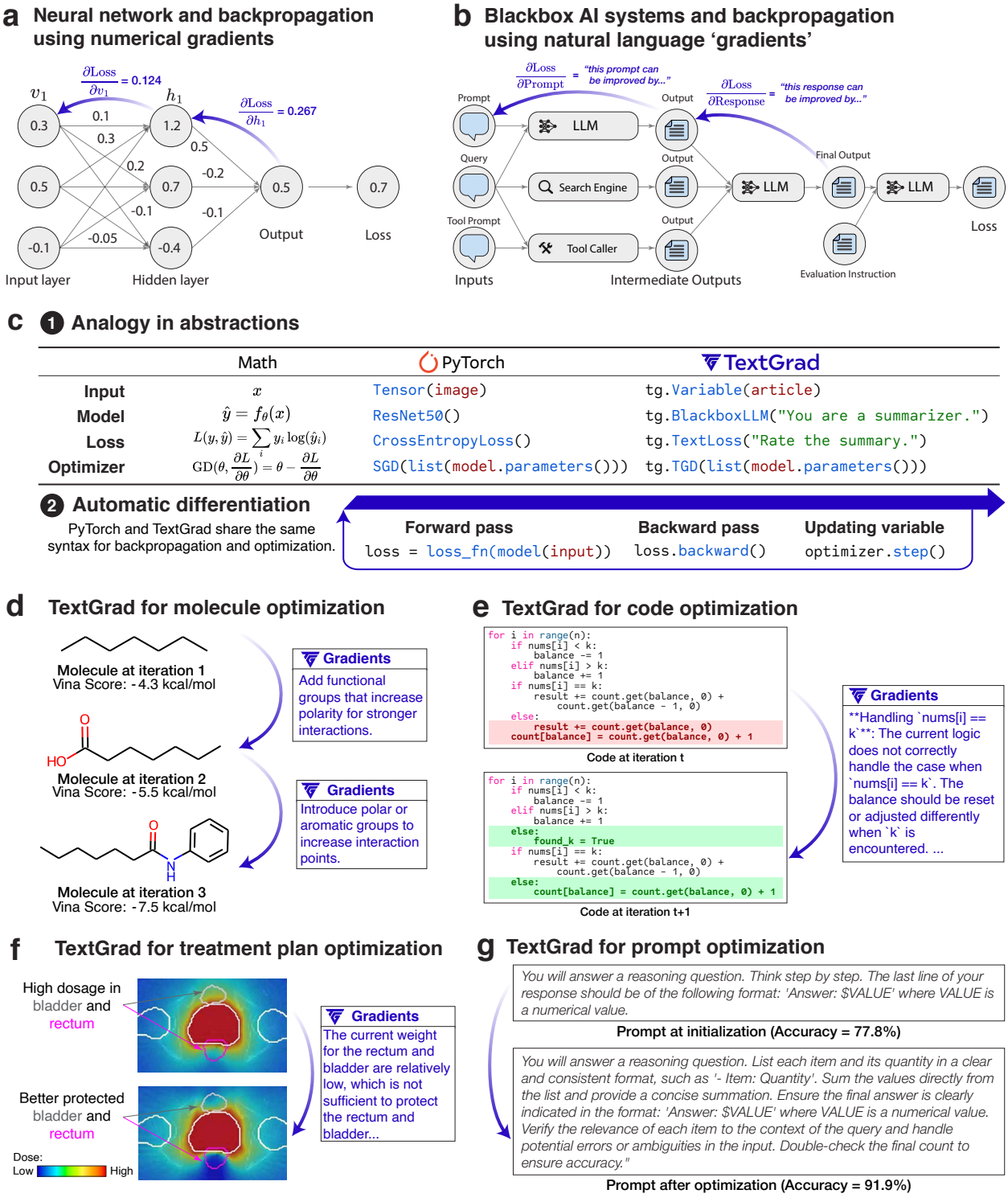


Figure 1: Automatic "Differentiation" via Text (a, b). Backpropagation of gradients is the driving force of deep learning. We do not have gradients for compound systems of blackbox AI systems, but we can construct analogous backpropagation for text-based feedback, forming the basis of TEXTGRAD. **Abstractions in TEXTGRAD (c).** We share the same abstractions and syntax as PyTorch to our framework generalizable and easy-to-learn. **Applications of TEXTGRAD (d,e,f,g).** In Section 3.4, we optimize molecular structures for properties such as druglikeness and protein binding affinity (d). In Section 3.1, we optimize solutions to coding problems (e). In Section 3.5, we optimize radiotherapy treatment plans to improve patient outcomes (f). In Section 3.3, we optimize prompts to improve the reasoning of language models (g).

where we denote the free parameter to optimize, the prompt, with **green**, and we use $+$ to denote concatenation of two strings, and use $\text{LLM}(x)$ to give x as a prompt to a language model to collect the response. We will use the chain notation:

$$\textbf{Prompt} + \text{Question} \xrightarrow{\text{LLM}} \text{Evaluation Instruction} + \text{Prediction} \xrightarrow{\text{LLM}} \text{Evaluation}, \tag{3}$$

as another way to denote this system, where we make one LLM call to generate a Prediction for a Question using a **Prompt**, and another LLM call to evaluate this Prediction.

Example gradient computation. In this example system, to improve the **Prompt** with respect to the evaluation, we instantiate an analog of the backpropagation [19] algorithm through the following:

Gradient computation for the simple graph: Prompt $\xrightarrow{\text{LLM}}$ Prediction $\xrightarrow{\text{LLM}}$ Evaluation

$$\frac{\partial \text{Evaluation}}{\partial \text{Prediction}} = \nabla_{\text{LLM}}(\text{Prediction}, \text{Evaluation}), \tag{4}$$

$$\frac{\partial \text{Evaluation}}{\partial \textbf{Prompt}} = \frac{\partial \text{Evaluation}}{\partial \text{Prediction}} \circ \frac{\partial \text{Prediction}}{\partial \textbf{Prompt}} = \nabla_{\text{LLM}}(\textbf{Prompt}, \text{Prediction}, \frac{\partial \text{Evaluation}}{\partial \text{Prediction}}), \tag{5}$$

where we use ∇_{LLM} for the gradient operator when the forward function is an LLM call.[a] In particular, this function returns natural language feedback such as '*This prediction can be improved by…*' where the feedback describes how to modify the variable to improve the downstream objective, analogous to gradients in optimization. We first collect feedback to the Prediction variable using the evaluation. Then, given this feedback and the (**Prompt** $\xrightarrow{\text{LLM}}$ Prediction) call, we collect the feedback on the **Prompt**.

[a] Note that we overload the notation to denote both for cases when the output variable does not have successors (e.g., $\nabla_{\text{LLM}}(\text{Prediction}, \text{Evaluation})$) and when it has successors, and thus gradients (e.g., $\nabla_{\text{LLM}}(\textbf{Prompt}, \text{Prediction}, \frac{\partial \text{Loss}}{\partial \text{Prediction}})$).

Below is a flexible way to instantiate ∇_{LLM} to collect feedback for a simple system like $x \xrightarrow{\text{LLM}} y \xrightarrow{\text{LLM}} \mathcal{L}$:

Example implementation for the gradient operator

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial x} = \nabla_{\text{LLM}}(x, y, \frac{\partial \mathcal{L}}{\partial y}) \triangleq\ & \texttt{"Here is a conversation with an LLM: \{x|y\}."} \\ & + \\ & \text{LLM}(\texttt{Here is a conversation with an LLM: \{x|y\}.} \\ & \quad \texttt{Below are the criticisms on \{y\}:} \\ & \quad \left\{\frac{\partial \mathcal{L}}{\partial y}\right\} \\ & \quad \texttt{Explain how to improve \{x\}.}),\end{aligned} \tag{6}$$

where the gradient object is a combination of the context in which the variable appears and the feedback obtained from an LLM[a], defined analogously to Pryzant *et al.* [25]. Note that this operator does not depend on, e.g., the application domain. Once implemented, the gradient operator for LLM calls are fixed throughout the framework for all applications.

[a] The exact prompts we use are different to ensure generality and flexibility; we use these examples only for exposition.

Example optimizer. In standard gradient descent, the current value of the variable is combined with the gradients through subtraction, e.g.:

$$\theta_{\text{new}} = \text{GradientDescent.step}(\theta, \frac{\partial \mathcal{L}}{\partial \theta}) = \theta - \frac{\partial \mathcal{L}}{\partial \theta}. \tag{7}$$

Continuing the gradient-based optimization analogy, we use *Textual* Gradient Descent (TGD):

Updating the **Prompt** variable in the simple graph via TGD.

$$\textbf{Prompt}_{\text{new}} = \text{TGD.step}(\textbf{Prompt}, \frac{\partial \text{Evaluation}}{\partial \textbf{Prompt}}). \tag{8}$$

to update the parameters, in this case, the **Prompt**. In particular, given the current variable and the gradients (feedback) we collected for this variable, the optimizer seeks to update this variable.

A concrete way to instantiate TGD is the following:

Example implementation of one TGD iteration

$$x_{\text{new}} = \text{TGD.step}(x, \frac{\partial \mathcal{L}}{\partial x}) \triangleq \text{LLM}(\texttt{Below are the criticisms on \{x\}:} \left\{\frac{\partial \mathcal{L}}{\partial x}\right\} \texttt{Incorporate the criticisms, and produce a new variable.}). \tag{9}$$

where x is the variable we would like to improve, and $\frac{\partial \mathcal{L}}{\partial x}$ is the feedback we obtained for the variable during the backward pass[a]. Similar to the gradient operator, this function also does not depend on the domain of application, and TGD implementation is the same across all uses of the framework.

[a] The exact prompts we use are different to ensure generality and flexibility; we use these examples only for exposition.

The general case

The abstraction readily applies to arbitrarily complex systems. Define a computation graph by

$$v = f_v(\text{PredecessorsOf}(v)) \quad \forall v \in \mathcal{V} \tag{10}$$

where v is a variable in the graph, $\mathcal{V}$ is the set of all variables in the graph, and SuccessorsOf returns the successors and PredecessorsOf returns the predecessors of a variable. Generally speaking, the value of v can be unstructured data, such as natural language text or images. For most of the results and exposition in this paper, v is natural language text.

Further, let us have f_v as the transformation that consumes a set of variables and produces the variable v. For instance, we can use an LLM or Numerical Simulator as a transformation. Since different functions will have different ways to compute gradients and collect feedback for, we will generally use ∇_f to denote the gradient function for a function f. For the sake of exposition, we will omit the subscript when the function is obvious.

The gradients are computed by

$$\frac{\partial \mathcal{L}}{\partial v} = \bigcup_{w \in \text{SuccessorsOf}(v)} \nabla_f \left(v, w, \frac{\partial \mathcal{L}}{\partial w}\right), \tag{11}$$

where we collect the set of gradients from all successors of v. Intuitively, we get feedback from every context in which a variable v was used, and aggregate them.

Equation 11 recursively computes the gradients of the downstream objective with respect to the desired variables v in the graph. The ∇_f function takes as input the gradients of $\mathcal{L}$ with respect to the successors of a given variable v, the value of the variable v and the successors themselves. Note that the final gradient variable comprises a set of contexts and criticisms for any place a variable was used in.

Finally, to update any desired variable v in the graph, we can use an optimizer:

$$v_{\text{new}} = \text{TGD.step}\left(v, \frac{\partial \mathcal{L}}{\partial v}\right), \tag{12}$$

which updates the value of v based on its current value and the gradients. For a computation graph where there are n edges, each iteration of optimization performs at most n additional language model calls to compute gradients (1 call using the gradient operator for each edge in the computation graph). For implementations of operations in **TEXTGRAD**, see Appendix A.

Objective functions

In numerical optimization and automatic differentiation, the objective function is typically a differentiable function, such as mean squared error or cross entropy. In **TEXTGRAD**, the objective can be a complex and potentially nondifferentiable function, where the domain and codomain of the function can be unstructured data. This choice adds important generality and flexibility to the framework. For instance, we demonstrate that the objective can be specified in natural language text and computed by prompting a language model (§3.3), an output of a code interpreter running unit tests (§3.1), or outputs of molecular simulation engines (§3.4). For instance, a simple loss function for a code snippet can be the following:

$$\begin{aligned}\text{Loss}(\textbf{code}, \text{target goal}) = \text{LLM}(&\text{Here is a code snippet:}\{\textbf{code}\}.\\ &\text{Here is the goal for this snippet:}\{\text{target goal}\}.\\ &\text{Evaluate the snippet for correctness and runtime complexity.}),\end{aligned} \tag{13}$$

where we can use this evaluation signal to optimize the code snippet, powered by the well-documented ability of LLMs to simulate human feedback, self-evaluate, and self-improve [10, 28–33].

Instance vs Prompt Optimization

There are two classes of optimization problems we explore.

In instance optimization, we directly treat a solution to a problem—e.g., a code snippet, the solution to a problem or a molecule—as an optimization variable. For instance, in Equation 13, we have a code instance that we would like to improve at test time. Our framework produces the gradients for and directly optimizes the **code** variable.

In prompt optimization, the goal is to find a prompt that improves the performance of an LLM across multiple queries for a task. For example, we may want to find a system prompt to an LLM that improve the performance on mathematical reasoning questions (see Section 3.3 for examples). In particular, we want the system prompt to generalize, in contrast to instance optimization where the only goal is to improve the solution for a given query at test time.

Crucially, both types of problems can be solved without hand-crafting the framework.

Optimization Techniques

Automatic differentiation is a strong analogy for **TEXTGRAD** and provides conceptual support for optimization techniques implemented in the framework. Below, we describe some examples.

Batch Optimization: We implement stochastic minibatch gradient descent [18, 34] for prompt optimization. Specifically, after doing a forward pass with multiple instances in a batch and evaluate individual loss

terms, we use the `tg.sum` function to sum the losses (akin to `torch.sum`). In the backward pass, gradients on variables through individual loss terms are concatenated, mirroring backpropagating through addition.

Constrained Optimization: We use natural language constraints, building on constrained optimization as an analogy [35]. In particular, we use natural language constraints (e.g., such as *'The last line of your response should be of the following format: 'Answer: $LETTER' where LETTER is one of ABCD."*) to guide the behavior of the optimizer. We observe that thanks to instruction-tuning [36, 37], language models can follow these simple constraints, although their reliability can reduce with too many constraints [38, 39].

Momentum: We use the analogy to momentum in gradient descent [40, 41]. When optimizing a variable, the TGD optimizer can optionally see the earlier iterations of the variable when making the update.
For more details on optimization techniques that are implemented in **TEXTGRAD**, see Appendix B.

3 Results

We demonstrate the flexibility of **TEXTGRAD** in a diverse array of applications. In § 3.1, we optimize code snippets to solve hard coding problems from LeetCode. In § 3.2, we optimize solutions to science questions. In § 3.3 we optimize prompts to improve the reasoning of LLMs. In § 3.4, we optimize chemical structures for improved molecular properties. In § 3.5, we optimize treatment plans for prostate cancer patients.

3.1 Code optimization

Code optimization is a hallmark use case of instance optimization. Here, the goal is to optimize some code to improve e.g., correctness or runtime complexity. We often have a computation graph like the following:

$$\text{Code-Refinement Objective} = \text{LLM}(\text{Problem} + \textbf{Code} + \text{Test-time Instruction} + \text{Local Test Results}) \quad (14)$$

where we optimize the **Code** to solve a given Problem with limited, local test supervision and self-evaluation through a test-instruction asking to critique the current iteration of the code. Figure 1e shows an example for the problem *You are given an array nums of size n consisting of distinct integers from 1 to n and a positive integer k. Return the number of non-empty subarrays in nums that have a median equal to k.* The first solution proposed by `gpt-4o` does not pass the tests. **TEXTGRAD** identifies an edge case in the first solution and provides a suggestion on how to improve it. The optimized implementation passes all tests.

Task: We use the LeetCode Hard dataset [26] to benchmark code optimization. LeetCode is an online platform that offers coding practice questions in preparation for technical interviews. The LeetCode Hard dataset contains examples of *hard* coding problems that are meant to be challenging for both humans and language models, where the success metric is Completion Rate, i.e., passing all test cases for a given problem (GPT-4 reportedly achieved a 7% completion rate [26]). LeetCode test cases are not public, and thus, after generation, the code has to be submitted to the LeetCode platform for evaluation on the unseen test cases. This makes the platform more suitable to evaluate the performance of language models.

Baseline: Reflexion [26] is the state-of-the-art method on the LeetCode Hard dataset. Their approach prompts an LLM to self-reflect on code snippets and the errors that were generated at test time using candidate unit tests. Given the self-reflection, the LLM is prompted again to provide an updated piece of code, conditioned on the self-reflection and the errors. We ran Reflexion on LeetCodeHard using `gpt-4o` using 1 in-context demonstration to guide the behavior (one-shot). In addition to Reflexion, we also run a zero-shot baseline using `gpt-4o` mimicking the same zero-shot baseline described in [26]. In comparison, **TEXTGRAD** runs in a zero-shot setting, without any demonstrations.

Results: Existing results [26] showed a 7% pass rate for GPT-4 zero-shot and 15% for GPT-4 with Reflexion. We show that these results have now been boosted to 23% for `gpt-4o` zero-shot and 31% when Reflexion is used. With **TEXTGRAD**, we can optimize solutions to achieve a performance of 36%. These improvements are more impressive considering that Reflexion was ran with in-context demonstrations and **TEXTGRAD** did not use any demonstrations (i.e. zero-shot).

Code Refinement Objective

LLM("You are an intelligent assistant used as an evaluator, and part of an optimization system. You will analyze a code implementation for a coding problem and unit test results. The code will be tested with harder tests, so do not just check if the code passes the provided tests. Think about the correctness of the code and its performance in harder test cases. Give very concise feedback. Investigate the code problem and the provided implementation. For each failed unit test case, start analyzing it by saying "The code did not pass this test because...". Explain why the current implementation is not getting the expected output. Do not provide a revised implementation. Carefully suggest why there are issues with the code and provide feedback.
{Test-time Instruction}
The coding problem:
{Problem}
Code generated that must be evaluated for correctness and runtime performance
{**Code**}
The test results:
{Local test Results}

Table 1: Code optimization for LeetCode Hard using `gpt-4o`. Results are averaged over 5 seeds.

Task	Method	Completion Rate
LeetCode Hard [26]	Zero-shot [26]	0.26
	Reflexion (1 demonstration, 5 iterations) [26]	0.31 ± 0.012
	TEXTGRAD (0 demonstrations, 5 iterations)	$\mathbf{0.36} \pm 0.018$

3.2 Solution optimization by test-time training to improve problem solving

Here, we focus on the task of *solution optimization* via **TEXTGRAD**. In solution optimization, the goal is to improve the solution to a complex problem, such as a question about quantum mechanics or organic chemistry. In particular, we often have a computation graph like the following:

$$\text{Solution Refinement Objective} = \text{LLM}(\text{Question} + \textbf{Solution} + \text{Test-time Instruction}) \tag{15}$$

where the parameter we optimize is the **Solution**, and the loss function is obtained by an evaluation of the solution, e.g., with an LLM. At each iteration, the LLM is prompted with the question, current solution, and some test-time instruction asking to critique or investigate the current iteration. Over the optimization trajectory, the solution is refined using this test-time self-evaluation. More generally, this idea is known as test-time training [42, 43], where a machine learning model is trained on a test instance at test-time, often with a self-supervised objective. Similarly, recent work have shown the merits of self-refinement also for reasoning tasks [26, 30, 44]. In particular, even though an LLM may not get the answer to a question or the solution to a problem right at first attempt, it can improve the response through iterative refinement.

For instance, the objective function for the refinement looks like the following:

Solution Refinement Objective

LLM("Below is a multi-choice question and a prediction. You are a critical and creative scientist. Your job is to investigate the prediction. Critically go through reasoning steps, and see if there is a reason why the prediction could be incorrect.
Use the Janusian Process, think about whether alternative answers could be true.
Question: {Question}
Answer by the language model: {**Solution**}")

Below is a representative implementation of solution optimization in TEXTGRAD.

```
# Assume we have the test_dataset
question, answer = test_dataset[i]
# Initialize the test time loss function
# This has the system prompt provided above as 'Solution Refinement Objective'
test_time_loss_fn = MultipleChoiceTestTime()
# Get a zero-shot solution from an LLM
solution: Variable = zero_shot_llm(question)
# Optimize the solution itself
optimizer = tg.TextualGradientDescent(parameters=[solution])

for iteration in range(max_iterations):
    optimizer.zero_grad()
    # Note how the loss is self-supervised (does not depend on any ground truth.)
    loss = test_time_loss_fn(question, solution)
    # Populate the gradients, and update the solution
    loss.backward()
    optimizer.step()
```

Code Snippet 1: A representative implementation of solution optimization in TEXTGRAD.

Task: Google-proof Question Answering (GPQA) [27] is a recent benchmark where challenging multiple-choice questions in physics, biology, and chemistry are created and labeled by domain experts who have or are pursuing PhD degrees. In this benchmark, experts and skilled non-experts are reported to achieve 81% and 22% accuracy respectively, demonstrating the difficulty of the questions. Importantly, this is a benchmark where performance has not yet saturated, where to our knowledge, the best reported results, achieved by `gpt-4o`, gets 53.6% accuracy in the diamond subset. We also use two challenging subsets (Machine Learning and College Physics) of the MMLU [45] question answering benchmark that is used to track the progress of language modeling and whether LLMs reached human-level performance. Here the expert human accuracy on average is around 90%. For the details of the question format and prompts, please see Appendix D.

Method: We report two baselines. First, the reported results in the `gpt-4o` release document states 53.6% accuracy. However, their official implementation uses a temperature of 0.5 for generations, thus we also test `gpt-4o` with temperature 0 and Chain-of-Thought (CoT) [46, 47] prompting provided in the official implementation[1]. For TEXTGRAD, we perform 3 iterations of test-time updates (i.e. update the solution three times) and perform majority voting across all solutions to get the final answer. We use string-based metrics to compute the final accuracy of each answer.

Results: With TEXTGRAD, we improve the performance of `gpt-4o` in the challenging question-answering tasks and report the results in Table 2. To our best knowledge, 55% is the best known result in the GPQA dataset so far. Similarly, we improve the performance in MMLU subsets from 85.7% to 88.4% (Machine Learning) and 91.2% to 95.1% (College Physics). These results show that by spending more computation at test-time through TEXTGRAD self-refinement, we can improve the question answering performance of even the most capable models.

3.3 Prompt optimization for reasoning

While LLMs demonstrate an impressive performance in reasoning tasks, their performance can be sensitive to the prompt used to guide their behavior. In particular, with the right choice of a prompt, their performance can be significantly improved [49]. In prompt optimization, the goal is to find a prompt or an instruction to guide the behavior of an LLM, such that it performs well on a given task. In particular, we often have a computation graph like the following:

$$\begin{aligned} \text{Answer} &= \text{LLM}(\textbf{Prompt}, \text{Question}) \\ \text{Evaluation Metric} &= \text{Evaluator}(\text{Answer}, \text{Ground Truth}) \end{aligned} \tag{16}$$

[1] We do this to minimize randomness, however, discussions claim there may be other sources of non-determinism with `gpt-4`.

Table 2: Solution optimization for zero-shot question answering with `gpt-4o`.

Dataset	Method	Accuracy (%)
Google-proof QA [27]	CoT [46, 47]	51.0
	Best reported [48]	53.6
	TEXTGRAD	**55.0**
MMLU-Machine Learning [45]	CoT [46, 47]	85.7
	TEXTGRAD	**88.4**
MMLU-College Physics [45]	CoT [46, 47]	91.2
	TEXTGRAD	**95.1**

where we have a Question for the task, an Answer to the question, and an Evaluation Metric indicating the quality of the output given the ground truth answer. For instance, for a question-answering task, the evaluation metric would be the accuracy of the answer, evaluated e.g. using string-based metrics.

Here, given a handful of training examples to optimize a **Prompt**, the goal is to maximize the performance of an LLM on the given task. In our experiments, our goal is to improve the performance of a weaker and cheaper model (e.g., `gpt-3.5-turbo-0125`) using the feedback generated by stronger models (e.g., `gpt-4o`). This is useful in practice because by paying a fixed cost to optimize a **Prompt**, the prompt-optimized weaker model can be used with cheaper inference costs instead of using the strong and more expensive model.

```
# Initialize the system prompt
system_prompt = tg.Variable("You are a helpful language model. Think step by step.",
                            requires_grad=True,
                            role_description="system prompt to the language model")

# Set up the model object 'parameterized by' the prompt.
model = tg.BlackboxLLM(system_prompt=system_prompt)

# Optimize the system prompt
optimizer = tg.TextualGradientDescent(parameters=[system_prompt])

for iteration in range(max_iterations):
    batch_x, batch_y = next(train_loader)
    optimizer.zero_grad()
    # Do the forward pass
    responses = model(batch_x)
    losses = [loss_fn(response, y) for (response, y) in zip(responses, batch_y)]
    total_loss = tg.sum(losses)
    # Perform the backward pass and compute gradients
    total_loss.backward()
    # Update the system prompt
    optimizer.step()
```

Code Snippet 2: A simple implementation of prompt optimization with TEXTGRAD.

Here, we use TEXTGRAD in a minibatch stochastic gradient descent setting [18, 34]. In particular, at each iteration, we use a few training examples to run the forward pass in Equation 16. A pseudocode and short implementation can be found in the snippet above. Full details of prompt optimization can be found in Appendix E. Unlike instance optimization, where TEXTGRAD tries to optimize each individual solution, the goal here is to optimize a single prompt that works well across all the questions in a benchmark.

Tasks: We explore prompt optimization in multiple datasets, including the two standard reasoning tasks (Object Counting and Word Sorting) from Big Bench Hard (randomly split into 50/100/100 train/validation/test samples) [50, 51], GSM8k grade-school math problem solving [52] (using train/validation/test splits from DSPy). For GSM8k and Object Counting, we use a string-based exact match metric to quantify accuracy (i.e. whether the final number provided in the response is the same as the ground truth answer).

For Word Sorting, we use an LLM to compare the response and the ground truth answer. We give more details about the tasks, prompts, and example queries in Appendix E.1.

Methods: We explore improving the performance of `gpt-3.5-turbo-0125` using `gpt-4o` [48] to provide feedback during backpropagation. In particular, while the forward model that performs the reasoning is `gpt-3.5-turbo-0125`, we use `gpt-4o` to provide the feedback and improve the prompt. We use a batch size of 3 with 12 iterations, i.e., the model sees 36 training examples in total, sampled randomly with replacement. After each iteration, we run a validation loop with the validation set of the datasets, and if the performance is better than the previous iteration we update the **Prompt**.

Baselines: We have two main baselines:

1. **Zero-shot Chain-of-Thought (CoT) [46, 47]:** We initialize all prompts as zero-shot CoT prompt, where a model is instructed to 'Think step-by-step' to explain its reasoning before giving its answer. This strategy is well-known to be a strong baseline for prompting.
2. **DSPy** is a state-of-the-art language model programming and prompt optimization framework [10], thus we use it as the reference baseline. We instantiate DSPy's BootstrappedFewShotRandomSearch (BFSR) optimizer with 10 candidate programs and 8 few-shot examples. This optimizer identifies demonstrations to include in the prompt as few-shot examples. This is done through generating traces of LLM inputs and outputs that individually pass the metric (in this case, accuracy) and includes CoT reasoning. It then applies random search over subsets of up to size eight shots with these demonstrations.

Table 3: Prompt optimization for reasoning tasks. With TEXTGRAD, we optimize a system prompt for `gpt-3.5-turbo` using `gpt-4o` as the gradient engine that provides the feedback during backpropagation.

Dataset	Method	Accuracy (%)
Object Counting [50, 51]	CoT (0-shot) [46, 47]	77.8
	DSPy (BFSR, 8 demonstrations) [10]	84.9
	TEXTGRAD (instruction-only, 0 demonstrations)	**91.9**
Word Sorting [50, 51]	CoT (0-shot) [46, 47]	76.7
	DSPy (BFSR, 8 demonstrations) [10]	**79.8**
	TEXTGRAD (instruction-only, 0 demonstrations)	**79.8**
GSM8k [52]	CoT (0-shot) [46, 47]	72.9
	DSPy (BFSR, 8 demonstrations) [10]	**81.1**
	TEXTGRAD (instruction-only, 0 demonstrations)	**81.1**

Example: TextGrad optimized prompt for `gpt-3.5-turbo-0125`

Prompt at initialization (GSM8k Accuracy= 72.9%):
You will answer a mathematical reasoning question. Think step by step. Always conclude the last line of your response should be of the following format: 'Answer: $VALUE' where VALUE is a numerical value."

Prompt after 12 iterations with batch size 3 (GSM8k Accuracy= 81.1%):
You will answer a mathematical reasoning question. Restate the problem in your own words to ensure understanding. Break down the problem into smaller steps, explaining each calculation in detail. Verify each step and re-check your calculations for accuracy. Use proper mathematical notation and maintain consistency with the context of the question. Always conclude with the final answer in the following format: 'Answer: $VALUE' where VALUE is a numerical value.

Results: Across all three tasks, TEXTGRAD improves the performance of the 0-shot prompt significantly. It performs similarly to DSPy [10] for Word Sorting and GSM8k, and improves over DSPy by 7% for Object Counting. While the 8 demonstrations in the context can help guide the behavior of the LLM, it can increase the cost of inference. Interestingly, the DSPy optimizer and TEXTGRAD make complementary adjustments—the former adds in-context demonstration examples and latter optimizes the system prompt. Adding the examples selected by DSPy to TEXTGRAD's optimized prompt could further improve perfor-

mance (for GSM8k, directly combining the demonstrations from DSPy with the instruction from TextGrad increases the accuracy to 82.1%), suggesting that a fruitful direction is to combine both approaches.

3.4 Molecule optimization

TEXTGRAD supports a variety of optimization problems, including multi-objective optimization tasks commonly found in science and engineering applications. For example, in drug discovery, researchers seek to discover or design molecules that maximize a variety of objectives with regards to synthesizability, efficacy, and safety [53, 54]. To demonstrate TEXTGRAD's applications to multi-objective optimization, we apply TEXTGRAD to drug molecule optimization, and show how our framework can interface with computational tools and optimize chemical structures towards simultaneously improving their binding affinity and druglikeness.

Task: A critical consideration for potential drug molecules is their *binding affinity*, which represents the strength of the interactions between the molecule and its protein target. Drug designers seek molecules with high binding affinities to relevant drug targets, as they require lower and less frequent doses to achieve efficacy. This affinity can be quantified by free energy ΔG, which describes the ratio of probabilities between bound and unbound ligand-receptor pairs. ΔG can be estimated using "docking" simulations of protein-ligand binding [55, 56]. In our experiments, we employ the Vina score from the Autodock Vina tool, a widely used physics-based docking simulator [57]. The more negative the Vina score, the greater probability that the drug will bind to its intended target.

Potential drug molecules are also evaluated by their *druglikeness*, which estimates how the molecule will behave *in vivo*, with respect to solubility, permeability, metabolic stability and transporter effects. Molecules with high druglikeness are more likely to be absorbed by the body and reach their targets [58]. One popular metric for "druglikeness" is the Quantatiative Estimate of Druglikeness (QED) score, a weighted composite metric of important chemical characteristics such as molecular weight, lipophilicity, polar surface area, among others. The QED score ranges from 0 to 1, where 1 indicates high druglikeness [59].

Though there are many more considerations for successful molecules, in our experiments, we restrict our objectives to minimizing the Vina score and maximizing the QED, due to the relative maturity of these two metrics. The competing tradeoffs between these two metrics makes the optimization task realistic and challenging. In particular, docking scores tend to prefer larger molecules with many functional groups that maximize interactions with a binding site [55, 56, 60]. In contrast, the druglikeness encourages lighter, simpler molecules that have better absorption properties [58, 59]. Thus, simultaneously optimizing both objectives is non-trivial.

Methods: We apply TEXTGRAD to drug molecule optimization by encoding molecules as SMILES strings and constructing a multi-objective loss from the Vina and QED scores. Namely, we perform instance optimization over SMILES strings, where the gradients generated by TEXTGRAD with respect to the multi-objective loss are used to update the text representing the molecule.

$$\text{Evaluation} = \text{LLM}((\texttt{Affinity}(\textbf{SMILES}_i, \text{target}), \texttt{Druglikeness}(\textbf{SMILES}_i)) \tag{17}$$

$$\textbf{SMILES}_{i+1} = \text{TGD.step}\left(\textbf{SMILES}_i, \frac{\partial \text{Evaluation}}{\partial \textbf{SMILES}_i}\right) \tag{18}$$

We use `gpt-4o` as our LLM with the prompt text found in Appendix F.2. At each iteration, the current molecule is evaluated by estimating its binding affinity to the target protein using the Vina score from Autodock Vina and using the QED score for druglikeness from RDKit (Appendix F.1). Each molecule is initialized as a small chemical fragment from a functional group. We apply TEXTGRAD to all 58 targets in the DOCKSTRING molecule evaluation benchmark [61]. These 58 targets consist of clinically relevant proteins sampled from a variety of structural classes, 29 of which have clinically approved drugs. For each target, we optimize a starting fragment using TEXTGRAD for 10 iterations, for 3 unique initial fragments. To evaluate our performance, we compare the characteristics of the molecules generated by TEXTGRAD to clinically approved drugs for the respective protein (Appendix F.3).

Results: For all 58 targets, TEXTGRAD consistently generates molecules with improved binding affinity and druglikeness irrespective of the initial starting fragment (Appendix F.4). For the 29 protein targets with

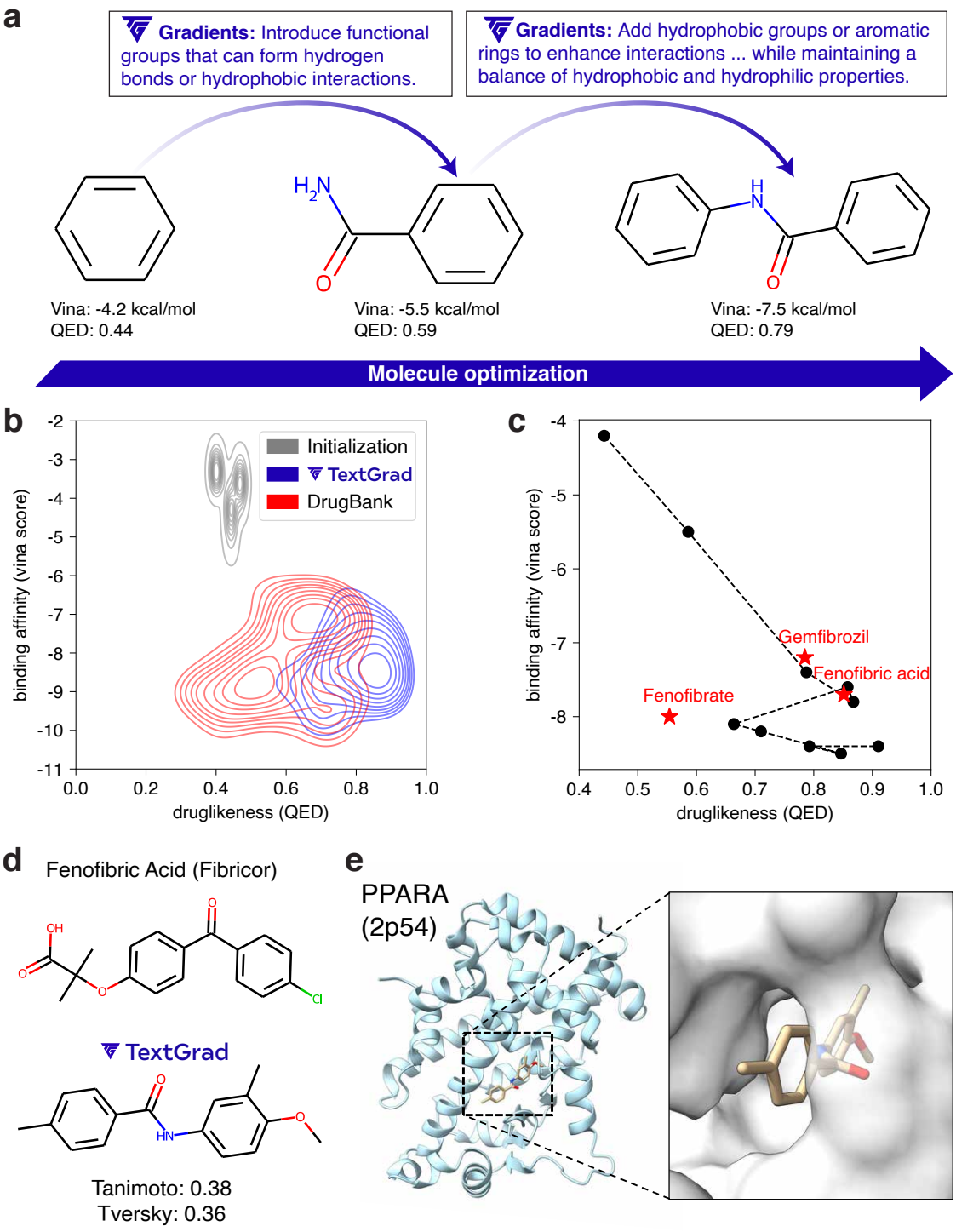


Figure 2: Molecule optimization via Text. TEXTGRAD optimizes a starting benzene fragment to improve its druglikeness (higher QED) and binding affinity (lower vina score) to the protein receptor PPARA. The textual gradients for the first three iterations are shown in (**a**), and the performance of all ten iterations compared to clinically approved molecules targetting PPARA in (**c**). The molecule at the final iteration has low structural similarity with its most similar clinically approved counterpart, and better QED and Vina scores **(d)** with a highly plausible pose geometry shown in **(e)**. Across 29 targets and three initial fragments, TextGrad successfully designs molecules with similar vina scores and greater QED scores than clinically approved molecules **(b)**.

clinically approved drugs, we observe that TEXTGRAD generates molecules with highly competitive affinity and druglikeness when compared to clinical molecules evaluated using the same loss function (Figure 2 **(b)**). The resulting molecules exhibit unique structures compared to their clinical approved counterparts and existing compounds (Appendix F.5), while maintaining similar *in silico* safety profiles (Appendix F.6). While there exist alternative machine learning methods for *de novo* molecular generation, TEXTGRAD offers two key advantages over its counterparts: Firstly, by combining traditonal chemoinformatics tools with the the general knowledge and reasoning capabilities of LLMs, TEXTGRAD produces competitive results even *without a prior training set*. Secondly, TEXTGRAD's framework of natural language gradients produce *explainable* decisions, enabling researchers to understand precisely how and why a molecule's structure was constructed. Together, these two characteristics invoke a promising future for the role of AI agents in scientific discovery.

3.5 Radiotherapy treatment plan optimization

Radiation therapy, also known as radiotherapy, is a cancer treatment that uses beams of intense energy, such as X-rays, to kill cancer cells. Before treatment begins, a radiotherapy team, including radiation oncologists and planners, collaborates to design an effective treatment plan. This involves determining the necessary dose of radiotherapy and pinpointing the exact locations that need treatment.

Radiotherapy treatment planning can be formulated as a two-loop optimization problem. The inner loop, known as inverse planning, includes processes such as influence map optimization and direct aperture optimization [62]. This optimization problem is typically a constrained one, solved by a numerical optimizer, aiming to minimize a weighted cost function balancing multiple conflicting objectives [63]. These objectives include delivering the prescribed dose to the planning target volume (PTV), which encompasses the tumor and an additional margin to account for uncertainties in planning or treatment delivery, while protecting critical normal tissues, known as organs at risk (OARs), from receiving unsafe doses.

The main challenge in treatment planning is translating overall clinical goals into weighted objective functions and dose constraints that yield an acceptable plan [62]. Human planners often use a trial-and-error approach, iteratively adjusting optimization hyperparameters based on the results of the optimization process until the plans meet clinical requirements [62]. These hyperparameters include the weights assigned to PTVs, organs, and other tissues in the objective function. This process can be subjective, influenced by the planner's experience and the available time, and involves repeatedly using computationally expensive optimization algorithms over many iterations. This makes the process inefficient, time-consuming, and costly [64].

Method. We apply TEXTGRAD to perform the outer loop optimization, i.e. hyperparameter optimization for the inner loop numerical optimizer. Instance optimization is performed with `gpt-4o` over the hyperparameters represented as a string: θ = "weight for PTV: [PTV WEIGHT], weight for bladder: [BLADDER WEIGHT], weight for rectum: [RECTUM WEIGHT], weight for femoral heads: [FH WEIGHT], weight for body: [BODY WEIGHT]". When hyperparameters are provided, we obtain the treatment plan by adopting a numerical optimizer and constructing a loss as the mismatch between the current plan and the clinical objectives. Specifically, to compute the gradient, we first solve the inner optimization loop using a numerical optimizer matRad[65] to obtain the corresponding treatment plan $P(\theta) = \texttt{matRad}(\theta)$. The loss is computed on the treatment plan P and the clinical goals g using an LLM with prompts provided in Section G.1.

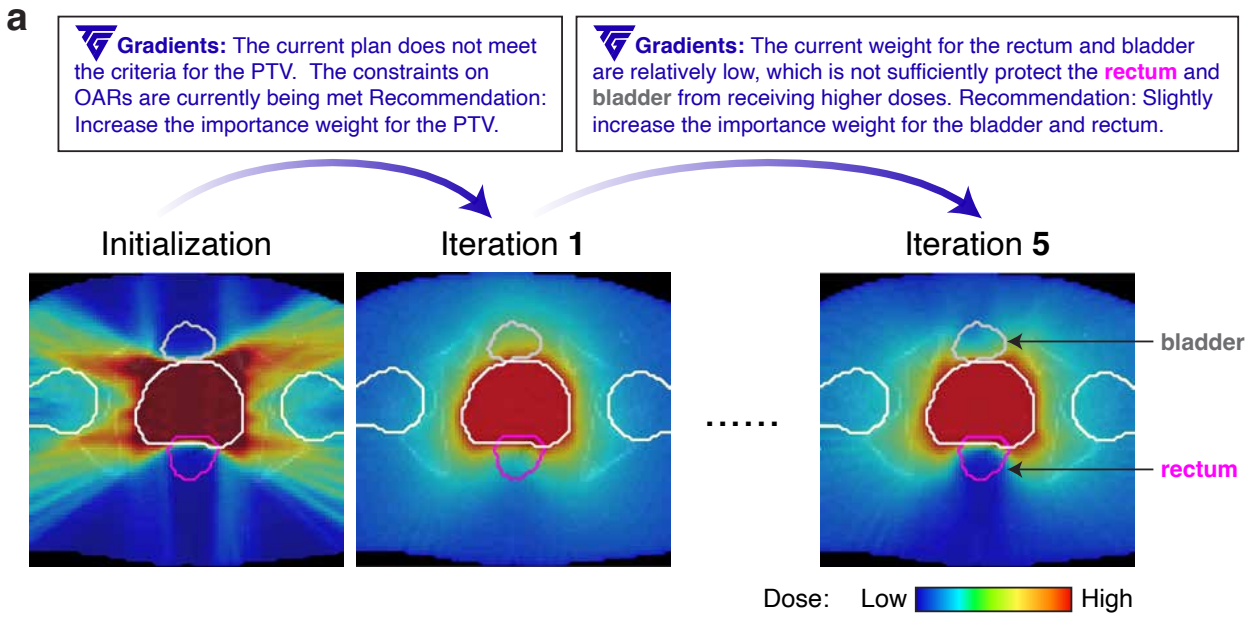
$$\mathcal{L} = \texttt{LLM}(P(\theta), g)$$

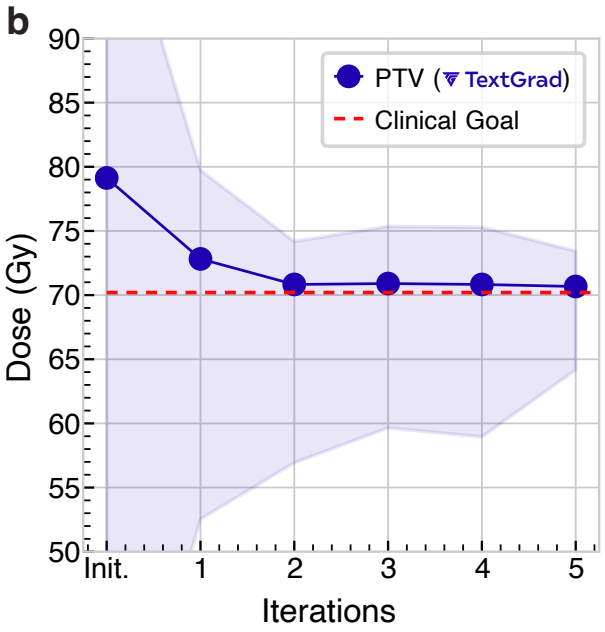
and the new hyperparameters are obtained by a TextGrad descent step

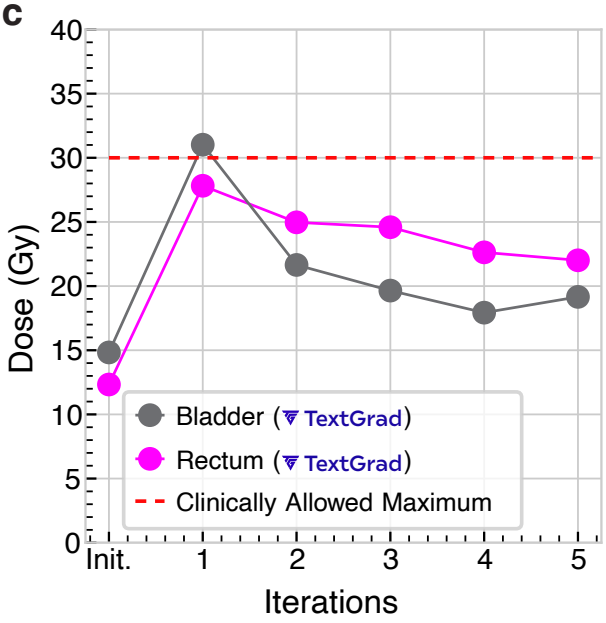
$$\theta_{\text{new}} = \text{TGD.step}\left(\theta, \frac{\partial \mathcal{L}}{\partial \theta}\right).$$

To further improve LLM's capability to understand the relationship between hyperparameters θ and the resulting plan P from matRad, a set of paired plans and their corresponding hyperparameters $\{(P_i, \theta_i)\}_{i=1}^N$ is provided as context for LLMs during the TGD.step. Therefore,

$$\theta_{\text{new}} = \text{TGD.step}\left(\theta, \frac{\partial \mathcal{L}}{\partial \theta} \middle| \{(P_i, \theta_i)\}_{i=1}^N\right).$$







d

Target	Method	Mean dose [Gy]	D_{95} [Gy]
PTV	Clinical Goal	70.20	70.20
	Radiation Oncologist	+1.97 (0.36)	-0.10 (0.15)
	TextGrad	**+0.51** (0.09)	**+0.00** (0.00)

e

Organ	Method	Mean dose [Gy]↓
Bladder	Radiation Oncologist	22.39 (5.55)
	TextGrad	**20.92** (0.79)
Rectum	Radiation Oncologist	23.88 (6.45)
	TextGrad	**17.18** (4.20)

Figure 3: Radiotherapy treatment plan optimization. Visualization of backpropagated textual gradients adjusting importance weights of planning target volumes (PTVs) and organs at risk (OARs) to balance tumor targeting and protection of OARs. Images in **(a)** show plan evolution from initialization to iteration 5. TEXTGRAD iteratively improves the mean dose and reduces the dose variance to the PTV, achieving the clinical goal over multiple iterations **(b)**. TEXTGRAD keeps the exposure for the bladder and rectum below clinically allowed maximums **(c)**. TEXTGRAD optimized plans have better dose metrics (mean dose and D_{95}) for PTV than clinically optimized plans **(d)**, and lower doses on bladder and rectum, indicating better protection of OARs **(e)**.

Evaluation metrics. To evaluate a treatment plan, we adopt several commonly used dose metrics as a plan cannot be evaluated using a single metric. We consider the mean dose delivered to the target/organ volume, as well as D_q, which denotes the minimum dose received by q% of the target/organ volume.

Results. The gradients generated by TEXTGRAD provide meaningful guidance to improve the hyperparameters. As illustrated in Figure 3, when there is dose spillage outside the Planning Target Volume (PTV), the gradient suggests an increase in the importance weight for the PTV. This adjustment results in a more uniform and confined dose for the PTV. However, this can lead to insufficient protection of the bladder and rectum as their relative weights are reduced. Therefore, in the following step, the gradients suggest slightly increasing the weights for the bladder and rectum, resulting in better protection for these organs. We compared TEXTGRAD optimized plans with the clinical plans used to treat five prostate cancer patients. In Figure 3 (c), we assess TextGrad's capabilities in achieving clinical goals for the PTV region. TextGrad outperforms the clinical plans across all metrics, achieving a higher mean dose, and a D_{95} that exactly matches the prescribed dose. In Figure 3 (d), we focus on the sparing of healthy organs. TextGrad-optimized plans achieve lower mean doses for these healthy organs, suggesting better organ sparing than the human-optimized plans. We report the averages across five plans and with standard deviation included in the bracket.

4 Related work

One related thread of work investigated the problem of prompt optimization. Practitioners demonstrated that prompt engineering strategies such as intelligently picking few-shot examples and in-context learning, CoT, ensembles can significantly boost performance of LLMs [66]. To automate this process, white-box methods that leverage numerical gradients were developed to optimize prompts [67–70], however, these methods cannot be used with closed-source models as they require access to model parameters. Various works investigated using LLMs as prompt optimizers [12, 25, 71].

Under prompt optimization, there are two works closest to our philosophy that have been our inspirations. First, DSPy [10, 72, 73] pioneered the idea of viewing complex LLM-based systems as programs with potentially many layers, and proposes ways to build and optimize them in a programmatic fashion. The framework is extensive, with results improving LLM performance in various question answering, reasoning, and prompt optimization tasks. Our work takes a different perspective that backpropagation and its extensions can be a general and powerful framework to optimize the new generation of AI systems, and perform multiple tasks outside of prompt optimization. In particular, we treat not only instructions or demonstrations as variables to optimize, but also the instances we care about themselves — such as molecules, treatment plans, code snippets, and so on. Second, greatly inspiring to us, Prompt Optimization with Textual Gradients (ProTeGi) [25] defines the Textual Gradients in the context of prompt optimization, where gradients are natural language feedback from LLMs given to the mistakes made during the task. While ProTeGi is built on the textual gradient analogy, we expand this analogy more broadly to automatic differentiation, and going substantially beyond prompt optimization tasks. In particular, both DSPy and ProTeGi focused on prompt optimization, while a significant advance of TEXTGRAD, as demonstrated through our diverse applications, is in instance optimization.

More generally, there is an emerging line of work built on the high-level idea of using LLMs as critics or optimizers [10, 12, 25, 26, 30, 71, 74–80]. While many of these earlier frameworks demonstrated the utility of LLMs as optimizers, we propose a single and general framework that was tested successfully in a variety of applications. Within this framework, we can reason about optimizing chains or stacks of LLMs [81–83]: we propagate natural language feedback. Similarly, once viewed as a general-purpose optimization engine, we can formulate many relevant problems instantiated as a few lines of code in our framework, such as test-time training [42, 43] or self-refinement of solutions and self-improvement [26, 30, 44, 84–91]. Building on the optimization analogy, we already transferred several analogies from the traditional optimization literature such as momentum [40] through using earlier iterations in the context, use of batch optimization [92], constrained optimization [35] using natural language constraints, and so on. Our work opens up a large space to design the new generation of optimization algorithms, all within the same framework.

5 Discussion

TextGrad is built on three key principles: i) It is a general and performant framework that is not hand-crafted for a specific application domain, ii) It is easy-to-use, mirroring PyTorch abstractions thus allowing knowledge transfer, iii) It is fully open-source. Through **TEXTGRAD**, we obtained state-of-the-art results in code optimization and PhD-level question answering, optimized prompts, and provided proof-of-concept results in scientific applications such as developing molecules and optimizing treatment plans.

While we took a first step, there are various limitations that motivate future work to realize the potential of automatic differentiation frameworks powered by LLMs. First, while we demonstrated the potential of backpropagating text feedback, there are many applications our framework can be extended to. We hope **TEXTGRAD** can be used to accelerate iterative processes in scientific discovery and increase the productivity of engineering efforts. For instance, to allow for this, we hope to extend the operations in our computation graphs to include more components used in practical LLM applications, such as for tool use [83] or retrieval-augmented generation systems [93]. Second, the automatic differentiation analogy enables a large design space for algorithms. We believe there are many fruitful connections to be drawn between numerical optimization, automatic differentiation, and **TEXTGRAD**. In particular, increasing the stability of the optimization using variance reduction techniques [94], adaptive gradients [95], or self-verification using LLMs [96] are interesting connections. Meta learning approaches [97–99] to optimize the TextGrad framework using methods such as TextGrad itself is also an intriguing direction of future work.

Finally, while we conducted proof-of-concept applications of **TEXTGRAD** to design new molecules and treatment plans with *in silico* validations, the ultimate test requires experimental and clinical assessments, which are outside of the scope of this paper.

As the paradigm of AI shifts from training individual models to optimizing compound systems involving multiple interacting LLM components and tools, we need a new generation of automated optimizers. **TEXTGRAD** combines the reasoning power of LLMs with the decomposable efficiency of backpropation to create a general framework to optimize AI systems.

Acknowledgements

We would like to thank Duygu Yilmaz, Begum Ergun, Fatih Dinc, Yu Sun, Omar Khattab, Ian Covert, Kyle Swanson, Omer Faruk Akgun, Yusuf Efe, Kevin Y Wu, Eric Wu, Kailas Vodrahalli, Oscar Pastor Serrano, Patrick John Chia, Jacopo Tagliabue, Nitya Thakkar, Elana Simon, Pan Lu, Sabri Eyuboglu, Irena Gao, Lingjiao Chen, and members of the Zou Group for their support and comments on this work.

References

1. Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., *et al.* Language models are few-shot learners. *Advances in neural information processing systems* **33,** 1877–1901 (2020).
2. Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., *et al.* Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
3. Reid, M., Savinov, N., Teplyashin, D., Lepikhin, D., Lillicrap, T., Alayrac, J.-b., Soricut, R., Lazaridou, A., Firat, O., Schrittwieser, J., *et al.* Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530* (2024).
4. AI@Meta. Llama 3 Model Card. https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md (2024).
5. Anthropic, A. The Claude 3 Model Family: Opus, Sonnet, Haiku. *Claude-3 Model Card* (2024).
6. Bommasani, R., Hudson, D. A., Adeli, E., Altman, R., Arora, S., von Arx, S., Bernstein, M. S., Bohg, J., Bosselut, A., Brunskill, E., *et al.* On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258* (2021).
7. Trinh, T. H., Wu, Y., Le, Q. V., He, H. & Luong, T. Solving olympiad geometry without human demonstrations. *Nature* **625,** 476–482 (2024).
8. Li, Y., Choi, D., Chung, J., Kushman, N., Schrittwieser, J., Leblond, R., Eccles, T., Keeling, J., Gimeno, F., Dal Lago, A., *et al.* Competition-level code generation with alphacode. *Science* **378,** 1092–1097 (2022).
9. Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K. & Press, O. *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering* 2024.
10. Khattab, O., Singhvi, A., Maheshwari, P., Zhang, Z., Santhanam, K., A, S. V., Haq, S., Sharma, A., Joshi, T. T., Moazam, H., Miller, H., Zaharia, M. & Potts, C. *DSPy: Compiling Declarative Language Model Calls into State-of-the-Art Pipelines* in *The Twelfth International Conference on Learning Representations* (2024). https://openreview.net/forum?id=sY5N0zY5Od.
11. Zaharia, M., Khattab, O., Chen, L., Davis, J. Q., Miller, H., Potts, C., Zou, J., Carbin, M., Frankle, J., Rao, N. & Ghodsi, A. *The Shift from Models to Compound AI Systems* https://bair.berkeley.edu/blog/2024/02/18/compound-ai-systems/. 2024.
12. Zhou, Y., Muresanu, A. I., Han, Z., Paster, K., Pitis, S., Chan, H. & Ba, J. *Large Language Models are Human-Level Prompt Engineers* in *The Eleventh International Conference on Learning Representations* (2023). https://openreview.net/forum?id=92gvk82DE-.
13. Krizhevsky, A., Sutskever, I. & Hinton, G. E. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems* **25** (2012).
14. Jumper, J., Evans, R., Pritzel, A., Green, T., Figurnov, M., Ronneberger, O., Tunyasuvunakool, K., Bates, R., Žídek, A., Potapenko, A., *et al.* Highly accurate protein structure prediction with AlphaFold. *Nature* **596,** 583–589 (2021).
15. Fawzi, A., Balog, M., Huang, A., Hubert, T., Romera-Paredes, B., Barekatain, M., Novikov, A., R Ruiz, F. J., Schrittwieser, J., Swirszcz, G., *et al.* Discovering faster matrix multiplication algorithms with reinforcement learning. *Nature* **610,** 47–53 (2022).
16. Mankowitz, D. J., Michi, A., Zhernov, A., Gelmi, M., Selvi, M., Paduraru, C., Leurent, E., Iqbal, S., Lespiau, J.-B., Ahern, A., *et al.* Faster sorting algorithms discovered using deep reinforcement learning. *Nature* **618,** 257–263 (2023).
17. Merchant, A., Batzner, S., Schoenholz, S. S., Aykol, M., Cheon, G. & Cubuk, E. D. Scaling deep learning for materials discovery. *Nature* **624,** 80–85 (2023).
18. Goodfellow, I., Bengio, Y. & Courville, A. *Deep learning* (MIT press, 2016).
19. Rumelhart, D. E., Hinton, G. E. & Williams, R. J. Learning representations by back-propagating errors. *Nature* **323,** 533–536 (1986).

20. Jia, Y., Shelhamer, E., Donahue, J., Karayev, S., Long, J., Girshick, R., Guadarrama, S. & Darrell, T. Caffe: Convolutional Architecture for Fast Feature Embedding. *arXiv preprint arXiv:1408.5093* (2014).
21. Bergstra, J., Breuleux, O., Bastien, F., Lamblin, P., Pascanu, R., Desjardins, G., Turian, J., Warde-Farley, D. & Bengio, Y. *Theano: A CPU and GPU Math Expression Compiler* in *Proceedings of the Python for Scientific Computing Conference (SciPy)* (2010).
22. Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., Devin, M., Ghemawat, S., Irving, G., Isard, M., *et al. TensorFlow: A System for Large-Scale Machine Learning* in *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)* (2016), 265–283.
23. Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., *et al.* Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems* **32** (2019).
24. Collobert, R., Bengio, S. & Mariéthoz, J. Torch: a modular machine learning software library (2002).
25. Pryzant, R., Iter, D., Li, J., Lee, Y., Zhu, C. & Zeng, M. *Automatic Prompt Optimization with "Gradient Descent" and Beam Search* in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing* (eds Bouamor, H., Pino, J. & Bali, K.) (Association for Computational Linguistics, Singapore, Dec. 2023), 7957–7968. https://aclanthology.org/2023.emnlp-main.494.
26. Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K. & Yao, S. *Reflexion: language agents with verbal reinforcement learning* in *Advances in Neural Information Processing Systems* **36** (2023). https://proceedings.neurips.cc/paper_files/paper/2023/file/1b44b878bb782e6954cd888628510e90-Paper-Conference.pdf.
27. Rein, D., Hou, B. L., Stickland, A. C., Petty, J., Pang, R. Y., Dirani, J., Michael, J. & Bowman, S. R. Gpqa: A graduate-level google-proof q&a benchmark. *arXiv preprint arXiv:2311.12022* (2023).
28. Li, X., Zhang, T., Dubois, Y., Taori, R., Gulrajani, I., Guestrin, C., Liang, P. & Hashimoto, T. B. *Alpacaeval: An automatic evaluator of instruction-following models* 2023.
29. Bai, Y., Jones, A., Ndousse, K., Askell, A., Chen, A., DasSarma, N., Drain, D., Fort, S., Ganguli, D., Henighan, T., *et al.* Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862* (2022).
30. Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., Alon, U., Dziri, N., Prabhumoye, S., Yang, Y., *et al.* Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems* **36** (2024).
31. Stiennon, N., Ouyang, L., Wu, J., Ziegler, D., Lowe, R., Voss, C., Radford, A., Amodei, D. & Christiano, P. F. Learning to summarize with human feedback. *Advances in Neural Information Processing Systems* **33,** 3008–3021 (2020).
32. Yuan, W., Pang, R. Y., Cho, K., Sukhbaatar, S., Xu, J. & Weston, J. Self-rewarding language models. *arXiv preprint arXiv:2401.10020* (2024).
33. Dubois, Y., Li, C. X., Taori, R., Zhang, T., Gulrajani, I., Ba, J., Guestrin, C., Liang, P. S. & Hashimoto, T. B. Alpacafarm: A simulation framework for methods that learn from human feedback. *Advances in Neural Information Processing Systems* **36** (2024).
34. Bottou, L. Large-scale machine learning with stochastic gradient descent. *Proceedings of COMPSTAT'2010,* 177–186 (2010).
35. Boyd, S., Boyd, S. P. & Vandenberghe, L. *Convex optimization* (Cambridge university press, 2004).
36. Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., *et al.* Training language models to follow instructions with human feedback. *Advances in neural information processing systems* **35,** 27730–27744 (2022).
37. Wei, J., Bosma, M., Zhao, V., Guu, K., Yu, A. W., Lester, B., Du, N., Dai, A. M. & Le, Q. V. *Finetuned Language Models are Zero-Shot Learners* in *International Conference on Learning Representations* (2022). https://openreview.net/forum?id=gEZrGCozdqR.

38. Yuksekgonul, M., Chandrasekaran, V., Jones, E., Gunasekar, S., Naik, R., Palangi, H., Kamar, E. & Nushi, B. *Attention Satisfies: A Constraint-Satisfaction Lens on Factual Errors of Language Models* in *The Twelfth International Conference on Learning Representations* (2024). https://openreview.net/forum?id=gfFVATffPd.
39. Abdin, M. I., Gunasekar, S., Chandrasekaran, V., Li, J., Yuksekgonul, M., Peshawaria, R. G., Naik, R. & Nushi, B. *KITAB: Evaluating LLMs on Constraint Satisfaction for Information Retrieval* in *The Twelfth International Conference on Learning Representations* (2024). https://openreview.net/forum?id=b3kDP3IytM.
40. Polyak, B. T. Some methods of speeding up the convergence of iteration methods. *USSR Computational Mathematics and Mathematical Physics* **4,** 1–17 (1964).
41. Sutskever, I., Martens, J., Dahl, G. & Hinton, G. *On the importance of initialization and momentum in deep learning* in *International conference on machine learning* (2013), 1139–1147.
42. Sun, Y., Wang, X., Liu, Z., Miller, J., Efros, A. & Hardt, M. *Test-Time Training with Self-Supervision for Generalization under Distribution Shifts* in *Proceedings of the 37th International Conference on Machine Learning* (PMLR, 2020). https://proceedings.mlr.press/v119/sun20b.html.
43. Sun, Y., Li, X., Dalal, K., Hsu, C., Koyejo, S., Guestrin, C., Wang, X., Hashimoto, T. & Chen, X. Learning to (learn at test time). *arXiv preprint arXiv:2310.13807* (2023).
44. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. R. & Cao, Y. *ReAct: Synergizing Reasoning and Acting in Language Models* in *The Eleventh International Conference on Learning Representations* (2023). https://openreview.net/forum?id=WE_vluYUL-X.
45. Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D. & Steinhardt, J. *Measuring Massive Multitask Language Understanding* in *International Conference on Learning Representations* (2021). https://openreview.net/forum?id=d7KBjmI3GmQ.
46. Kojima, T., Gu, S. S., Reid, M., Matsuo, Y. & Iwasawa, Y. Large language models are zero-shot reasoners. *Advances in neural information processing systems* **35,** 22199–22213 (2022).
47. Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., Zhou, D., *et al.* Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* **35,** 24824–24837 (2022).
48. OpenAI. *Hello GPT-4o* Accessed: 2024-05-18. 2024. https://openai.com/index/hello-gpt-4o/.
49. Liu, P., Yuan, W., Fu, J., Jiang, Z., Hayashi, H. & Neubig, G. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing. *ACM Computing Surveys* **55,** 1–35 (2023).
50. Suzgun, M., Scales, N., Schärli, N., Gehrmann, S., Tay, Y., Chung, H. W., Chowdhery, A., Le, Q., Chi, E., Zhou, D. & Wei, J. *Challenging BIG-Bench Tasks and Whether Chain-of-Thought Can Solve Them* in *Findings of the Association for Computational Linguistics: ACL 2023* (Association for Computational Linguistics, Toronto, Canada, July 2023). https://aclanthology.org/2023.findings-acl.824.
51. Srivastava, A. *et al.* Beyond the Imitation Game: Quantifying and extrapolating the capabilities of language models. *Transactions on Machine Learning Research.* ISSN: 2835-8856. https://openreview.net/forum?id=uyTL5Bvosj (2023).
52. Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C. & Schulman, J. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168* (2021).
53. Nicolaou, C. A. & Brown, N. Multi-objective optimization methods in drug design. *Drug Discovery Today: Technologies* **10,** e427–e435 (2013).
54. Hoelder, S., Clarke, P. A. & Workman, P. Discovery of small molecule cancer drugs: successes, challenges and opportunities. *Molecular oncology* **6,** 155–176 (2012).
55. Kontoyianni, M. Docking and virtual screening in drug discovery. *Proteomics for drug discovery: Methods and protocols,* 255–266 (2017).

56. Agarwal, S. & Mehrotra, R. An overview of molecular docking. *JSM chem* **4,** 1024–1028 (2016).
57. Trott, O. & Olson, A. J. AutoDock Vina: improving the speed and accuracy of docking with a new scoring function, efficient optimization, and multithreading. *Journal of computational chemistry* **31,** 455–461 (2010).
58. Ursu, O., Rayan, A., Goldblum, A. & Oprea, T. I. Understanding drug-likeness. *Wiley Interdisciplinary Reviews: Computational Molecular Science* **1,** 760–781 (2011).
59. Bickerton, G. R., Paolini, G. V., Besnard, J., Muresan, S. & Hopkins, A. L. Quantifying the chemical beauty of drugs. *Nature chemistry* **4,** 90–98 (2012).
60. Bender, B. J., Gahbauer, S., Luttens, A., Lyu, J., Webb, C. M., Stein, R. M., Fink, E. A., Balius, T. E., Carlsson, J., Irwin, J. J., *et al.* A practical guide to large-scale docking. *Nature protocols* **16,** 4799–4832 (2021).
61. García-Ortegón, M., Simm, G. N., Tripp, A. J., Hernández-Lobato, J. M., Bender, A. & Bacallado, S. DOCKSTRING: easy molecular docking yields better benchmarks for ligand design. *Journal of chemical information and modeling* **62,** 3486–3502 (2022).
62. Khan, F. M., Sperduto, P. W. & Gibbons, J. P. *Khan's Treatment Planning in Radiation Oncology:*. (Lippincott Williams & Wilkins, 2021).
63. Webb, S. The physical basis of IMRT and inverse planning. *The British journal of radiology* **76,** 678–689 (2003).
64. Hussein, M., Heijmen, B. J. M., Verellen, D. & Nisbet, A. Automation in Intensity Modulated Radiotherapy Treatment Planning—a Review of Recent Innovations. *British Journal of Radiology* **91,** 20180270. ISSN: 0007-1285. (2024) (Dec. 2018).
65. Wieser, H.-P., Cisternas, E., Wahl, N., Ulrich, S., Stadler, A., Mescher, H., Müller, L.-R., Klinge, T., Gabrys, H., Burigo, L., *et al.* Development of the open-source dose calculation and optimization toolkit matRad. *Medical Physics* **44,** 2556–2568 (2017).
66. Nori, H., Lee, Y. T., Zhang, S., Carignan, D., Edgar, R., Fusi, N., King, N., Larson, J., Li, Y., Liu, W., *et al.* Can generalist foundation models outcompete special-purpose tuning? case study in medicine. *arXiv preprint arXiv:2311.16452* (2023).
67. Shin, T., Razeghi, Y., Logan IV, R. L., Wallace, E. & Singh, S. *AutoPrompt: Eliciting Knowledge from Language Models with Automatically Generated Prompts* in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (Association for Computational Linguistics, Online, Nov. 2020), 4222–4235. https://aclanthology.org/2020.emnlp-main.346.
68. Jia, M., Tang, L., Chen, B.-C., Cardie, C., Belongie, S., Hariharan, B. & Lim, S.-N. *Visual prompt tuning* in *European Conference on Computer Vision* (2022), 709–727.
69. Li, X. L. & Liang, P. *Prefix-Tuning: Optimizing Continuous Prompts for Generation* in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)* (Association for Computational Linguistics, Online, Aug. 2021), 4582–4597. https://aclanthology.org/2021.acl-long.353.
70. Chen, X., Zhang, N., Xie, X., Deng, S., Yao, Y., Tan, C., Huang, F., Si, L. & Chen, H. *Knowprompt: Knowledge-aware prompt-tuning with synergistic optimization for relation extraction* in *Proceedings of the ACM Web conference 2022* (2022), 2778–2788.
71. Ye, Q., Axmed, M., Pryzant, R. & Khani, F. Prompt engineering a prompt engineer. *arXiv preprint arXiv:2311.05661* (2023).
72. Khattab, O., Santhanam, K., Li, X. L., Hall, D., Liang, P., Potts, C. & Zaharia, M. Demonstrate-Search-Predict: Composing Retrieval and Language Models for Knowledge-Intensive NLP. *arXiv preprint arXiv:2212.14024* (2022).
73. Singhvi, A., Shetty, M., Tan, S., Potts, C., Sen, K., Zaharia, M. & Khattab, O. DSPy Assertions: Computational Constraints for Self-Refining Language Model Pipelines. *arXiv preprint arXiv:2312.13382* (2023).

74. Yang, C., Wang, X., Lu, Y., Liu, H., Le, Q. V., Zhou, D. & Chen, X. *Large Language Models as Optimizers* in *The Twelfth International Conference on Learning Representations* (2024). https://openreview.net/forum?id=Bb4VGOWELI.

75. Song, X., Tian, Y., Lange, R. T., Lee, C., Tang, Y. & Chen, Y. *Position: Leverage Foundational Models for Black-Box Optimization* 2024. arXiv: 2405.03547 [cs.LG].

76. Liu, T., Astorga, N., Seedat, N. & van der Schaar, M. *Large Language Models to Enhance Bayesian Optimization* in *The Twelfth International Conference on Learning Representations* (2024). https://openreview.net/forum?id=OOxotBmGol.

77. Wang, R., Zelikman, E., Poesia, G., Pu, Y., Haber, N. & Goodman, N. *Hypothesis Search: Inductive Reasoning with Language Models* in *The Twelfth International Conference on Learning Representations* (2024). https://openreview.net/forum?id=G7UtIGQmjm.

78. Gao, L., Dai, Z., Pasupat, P., Chen, A., Chaganty, A. T., Fan, Y., Zhao, V., Lao, N., Lee, H., Juan, D.-C. & Guu, K. *RARR: Researching and Revising What Language Models Say, Using Language Models* in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (Association for Computational Linguistics, Toronto, Canada, July 2023), 16477–16508. https://aclanthology.org/2023.acl-long.910.

79. Chen, X., Lin, M., Schärli, N. & Zhou, D. *Teaching Large Language Models to Self-Debug* in *The Twelfth International Conference on Learning Representations* (2024). https://openreview.net/forum?id=KuPixIqPiq.

80. Shypula, A. G., Madaan, A., Zeng, Y., Alon, U., Gardner, J. R., Yang, Y., Hashemi, M., Neubig, G., Ranganathan, P., Bastani, O. & Yazdanbakhsh, A. *Learning Performance-Improving Code Edits* in *The Twelfth International Conference on Learning Representations* (2024). https://openreview.net/forum?id=ix7rLVHXyY.

81. Chase, H. *LangChain* Oct. 17, 2022. https://github.com/langchain-ai/langchain.

82. Liu, J. *LlamaIndex* https://github.com/jerryjliu/llamaindex. 2022.

83. Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N. & Scialom, T. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems* **36** (2024).

84. Press, O., Zhang, M., Min, S., Schmidt, L., Smith, N. & Lewis, M. *Measuring and Narrowing the Compositionality Gap in Language Models* in *Findings of the Association for Computational Linguistics: EMNLP 2023* (Association for Computational Linguistics, Singapore, Dec. 2023), 5687–5711. https://aclanthology.org/2023.findings-emnlp.378.

85. Zelikman, E., Wu, Y., Mu, J. & Goodman, N. *STaR: Bootstrapping Reasoning With Reasoning* in *Advances in Neural Information Processing Systems* (eds Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K. & Oh, A.) **35** (Curran Associates, Inc., 2022), 15476–15488. https://proceedings.neurips.cc/paper_files/paper/2022/file/639a9a172c044fbb64175b5fad42e9a5-Paper-Conference.pdf.

86. Huang, J., Gu, S., Hou, L., Wu, Y., Wang, X., Yu, H. & Han, J. *Large Language Models Can Self-Improve* in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing* (eds Bouamor, H., Pino, J. & Bali, K.) (Association for Computational Linguistics, Singapore, Dec. 2023). https://aclanthology.org/2023.emnlp-main.67.

87. Yang, K., Tian, Y., Peng, N. & Klein, D. *Re3: Generating Longer Stories With Recursive Reprompting and Revision* in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing* (Association for Computational Linguistics, Dec. 2022). https://aclanthology.org/2022.emnlp-main.296.

88. Xie, Y., Kawaguchi, K., Zhao, Y., Zhao, X., Kan, M.-Y., He, J. & Xie, Q. *Self-Evaluation Guided Beam Search for Reasoning* in *Thirty-seventh Conference on Neural Information Processing Systems* (2023). https://openreview.net/forum?id=Bw82hwg5Q3.

89. Paul, D., Ismayilzada, M., Peyrard, M., Borges, B., Bosselut, A., West, R. & Faltings, B. *REFINER: Reasoning Feedback on Intermediate Representations* in *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)* (Association for Computational Linguistics, Mar. 2024). https://aclanthology.org/2024.eacl-long.67.

90. Zhao, A., Huang, D., Xu, Q., Lin, M., Liu, Y.-J. & Huang, G. *Expel: Llm agents are experiential learners* in *Proceedings of the AAAI Conference on Artificial Intelligence* **38** (2024), 19632–19642.

91. Le, H., Chen, H., Saha, A., Gokul, A., Sahoo, D. & Joty, S. *CodeChain: Towards Modular Code Generation Through Chain of Self-revisions with Representative Sub-modules* in *The Twelfth International Conference on Learning Representations* (2024). https://openreview.net/forum?id=vYhglxSj8j.

92. Robbins, H. & Monro, S. A stochastic approximation method. *The Annals of Mathematical Statistics,* 400–407 (1951).

93. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., *et al.* Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems* **33,** 9459–9474 (2020).

94. Robert, C. P. & Casella, G. Simulation and the Monte Carlo method. *Springer Texts in Statistics, New York: Springer* **2** (1999).

95. Kingma, D. & Ba, J. *Adam: A Method for Stochastic Optimization* in *International Conference on Learning Representations (ICLR)* (San Diega, CA, USA, 2015).

96. Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I. & Cobbe, K. *Let's Verify Step by Step* in *The Twelfth International Conference on Learning Representations* (2024). https://openreview.net/forum?id=v8L0pN6EOi.

97. Schmidhuber, J. *Evolutionary Principles in Self-Referential Learning* PhD thesis (Technical University of Munich, 1987).

98. Thrun, S. & Pratt, L. in *Learning to Learn* 3–17 (Springer, 1998).

99. Finn, C., Abbeel, P. & Levine, S. *Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks* in *Proceedings of the 34th International Conference on Machine Learning* (2017).

100. *RDKit: Open-source cheminformatics* http://www.rdkit.org.

101. Hughes, J. P., Rees, S., Kalindjian, S. B. & Philpott, K. L. Principles of early drug discovery. *British journal of pharmacology* **162,** 1239–1249 (2011).

102. Wenlock, M. C., Austin, R. P., Barton, P., Davis, A. M. & Leeson, P. D. A comparison of physiochemical property profiles of development and marketed oral drugs. *Journal of medicinal chemistry* **46,** 1250–1256 (2003).

103. Knox, C., Wilson, M., Klinger, C. M., Franklin, M., Oler, E., Wilson, A., Pon, A., Cox, J., Chin, N. E., Strawbridge, S. A., *et al.* Drugbank 6.0: the drugbank knowledgebase for 2024. *Nucleic Acids Research* **52,** D1265–D1275 (2024).

104. Berry, M., Fielding, B. & Gamieldien, J. Practical considerations in virtual screening and molecular docking. *Emerging trends in computational biology, bioinformatics, and systems biology,* 487 (2015).

105. Birhane, A., Kasirzadeh, A., Leslie, D. & Wachter, S. Science in the age of large language models. *Nature Reviews Physics* **5,** 277–280 (2023).

106. Nikolova, N. & Jaworska, J. Approaches to measure chemical similarity–a review. *QSAR & Combinatorial Science* **22,** 1006–1026 (2003).

107. Gaulton, A., Bellis, L. J., Bento, A. P., Chambers, J., Davies, M., Hersey, A., Light, Y., McGlinchey, S., Michalovich, D., Al-Lazikani, B., *et al.* ChEMBL: a large-scale bioactivity database for drug discovery. *Nucleic acids research* **40,** D1100–D1107 (2012).

108. Swanson, K., Walther, P., Leitz, J., Mukherjee, S., Wu, J. C., Shivnaraine, R. V. & Zou, J. ADMET-AI: A machine learning ADMET platform for evaluation of large-scale chemical libraries. *bioRxiv,* 2023–12 (2023).

109. Xing, L., Li, J. G., Donaldson, S., Le, Q. T. & Boyer, A. L. Optimization of Importance Factors in Inverse Planning. *Physics in Medicine & Biology* **44,** 2525. ISSN: 0031-9155. (2024) (Oct. 1999).

A TEXTGRAD Details

A.1 Variables

`Variables` are the nodes in the computation graph. Below are the most important attributes of `Variables`:

1. **Value** is the unstructured data that the variable contains. Throughout this manuscript, all values are text data.
2. **Role description** is an informative string that describes the role of the variable in the computation graph. We use these roles to let the user inject knowledge into the graph and guide the optimization behavior. More information is described below.
3. **Gradients** are the natural language feedback provided by the LLMs during the backward pass. These describe the changes to make the variable so that the downstream loss can be improved. For an example backward operation that populates gradients, please read Section A.3.
4. **Predecessors** are the set of variables that are used to generate a given variable. For instance, if we are giving an instruction to an LLM and getting a response, the instruction would be the predecessor of the response. During the backward pass, the gradients on the successor are passed to its predecessors, to provide guidance around how to improve the downstream objective.
5. **Requires grad** indicates whether or not the gradients will be populated during the backward pass, analogous to PyTorch. For instance, if the user does not wish to compute gradients for a question, then simply write `Variable(value=question, requires_grad=False, ...)` to indicate this.

Role Description: In TEXTGRAD, each variable has a *role description*. In particular, this is a string that describes the role of the variable in the computation graph, such as *system prompt to the language model* or *prediction by the language model*. While sometimes populated automatically, in general role descriptions are one of the primary ways to inject user knowledge into the optimization process.

Empirically, we find that role descriptions can significantly steer the optimization process. For instance, setting the role of a prediction to be *the final numerical answer to the language model* guides the Textual Gradient Descent optimizer, that prompts a language model to update the value of the variable using the feedback, to update the variable such that it is only a numerical value. In comparison, a role description such as *the reasoning for the solution and the final prediction* guides the optimizer to produce the reasoning along with the final prediction.
Here is an example usage:

```
import textgrad as tg

system_prompt = tg.Variable("You will be given a question and think step-by-step.
", requires_grad=True, role_description="system prompt to the language model that
 will be reused across queries")

model = tg.BlackBoxLLM(system_prompt=system_prompt)
```

Code Snippet 3: An example usage of a role description.

A.2 Backpropagation

The backpropagation algorithm in TEXTGRAD mirrors existing autograd frameworks for deep learning. See Algorithm 1 for the pseudocode.

A.3 Functions

TEXTGRAD offers several operations where both the forward and backward computations are defined – as such, these operations are composable. The abstract `textgrad.autograd.Function` class has two methods:

Algorithm 1 Backpropagation in TEXTGRAD

Input: Variables $v \in \mathcal{V}$ in a graph, Loss variable $\mathcal{L}$, Backward Engine (LLM) $\mathcal{M}$ that will provide textual gradients
`# Initializing gradients`
for each $v \in \mathcal{V}$ **do**
  v.gradients $= \emptyset$
end for
`# Topological Sorting`
$Q \leftarrow$ TopologicalSort(G)
`# Backpropagation`
for v in Q **do**
  `# Populate gradients in predecessors`
  for each $u \in$ PredecessorsOf(v) **do**
    `#` Here, we are omitting subscript v in f. Semantically, f is the function that generates v, and ∇_f is the backward operation for that function.
    `# Semantically, this provides feedback to the variable` u, given how v is produced, and the feedback we already collected for v.
    u.gradients.add $\left(\nabla_f\left(u, v, \frac{\partial \mathcal{L}}{\partial v}\right)\right)$
  end for
end for

`forward` and `backward`, mirroring the PyTorch syntax. Each function has to define both of these methods. Below, we describe a couple of the most used operations in this paper.

`LLMCall` **Function:** Currently, the most crucial operation in TEXTGRAD is the call to language models.

Forward mode. The forward mode is simple: We make a call to an LLM, through an API or through the local machine. When a call is made, all the input variables are registered as the predecessors of the response from the LLM. For instance, if we ask a question to an LLM using an `instruction` and a `question` variable, the `response` variable's predecessors will be `[instruction, question]`. When doing the backward pass, the gradients on the `response` will be backpropagated to `question` and `instruction`.

Backward mode. To ensure the backward function runs smoothly and generally, we add the following glossary to the system prompt:

Glossary for Backward Mode of the `LLMCall` function

```
### Glossary of tags that will be sent to you:
# - <LM_SYSTEM_PROMPT>: The system prompt for the language model.
# - <LM_INPUT>: The input to the language model.
# - <LM_OUTPUT>: The output of the language model.
# - <OBJECTIVE_FUNCTION>: The objective of the optimization task.
# - <VARIABLE>: Specifies the span of the variable.
# - <ROLE>: The role description of the variable.
```

Using these tags, the LLM is made aware of the objective, the role and the value of the variable to give feedback to, and the full conversation in the forward mode.

This glossary is provided in the system prompt to the backward engine LLM:

System prompt for the backward mode of the `LLMCall` function

You are part of an optimization system that improves a given text (i.e. the variable). You are the gradient (feedback) engine. Your only responsibility is to give intelligent and creative feedback and constructive criticism to variables, given an objective specified in <OBJECTIVE_FUNCTION </OBJECTIVE_FUNCTION> tags. The variables may be solutions to problems, prompts to language models, code, or any other text-based variable. Pay attention to the role description of the variable, and the context in which it is used. You should assume that the variable will be used in a similar context in the future. Only provide strategies, explanations, and methods to change in the variable. DO NOT propose a new version of the variable, that will be the job of the optimizer. Your only job is to send feedback and criticism (compute 'gradients'). For instance, feedback can be in the form of 'Since language models have the X failure mode...', 'Adding X can fix this error because...', 'Removing X can improve the objective function because...', 'Changing X to Y would fix the mistake ...', that gets at the downstream objective.
If a variable is already working well (e.g. the objective function is perfect, an evaluation shows the response is accurate), you should not give feedback.
{GLOSSARY}

Most of this setup aims to ensure that the user does not have to modify the gradient computation. All of our experiments in the diverse set of applications are done with the same backward mode.

For instance, an example prompt for the gradient computation looks like the following:

An example backward mode prompt for the `LLMCall` function

You will give feedback to a variable with the following role: <ROLE> system prompt to a language model </ROLE>. Here is a conversation with a language model (LM):
<LM_SYSTEM_PROMPT> You will answer a reasoning question. Think step by step. Always conclude the last line of your response should be of the following format: 'Answer: $VALUE' where VALUE is a numerical value. </LM_SYSTEM_PROMPT>
<LM_INPUT> I have a chicken, a frog, a mouse, a cat, two pigs, and two rabbits. How many animals do I have? </LM_INPUT>
<LM_OUTPUT> To find the total number of animals, we simply add up the number of each type of animal: 1 chicken + 1 frog + 1 mouse + 1 cat + 2 pigs + 2 rabbits = 8 animals.
Answer: 8 </LM_OUTPUT>
This conversation is part of a larger system. The <LM_OUTPUT> was later used as response from the language model.
<OBJECTIVE_FUNCTION>Your goal is to give feedback to the variable to address the following feedback on the LM_OUTPUT: To improve the runtime of the string-based function that checks if the prediction is correct, consider the following feedback:
1. **Simplify the Response**: For example, instead of "To find the total number of animals, we simply add (...) + 2 pigs + 2 rabbits = 8 animals. Answer: 8", a more concise response like "Total animals: 8" would be more efficient.
...
By implementing these strategies, the response from the language model can be optimized to improve the runtime of the string-based function that checks if the prediction is correct. </OBJECTIVE_FUNCTION>
We are interested in giving feedback to the system prompt to a language model. Specifically, give feedback to the following span of text:
<VARIABLE> You will answer a reasoning question. Think step by step. (...) following format: 'Answer: $VALUE' where VALUE is a numerical value. </VARIABLE>
Given the above history, describe how the system prompt to a language model could be improved to improve the <OBJECTIVE_FUNCTION>. Be very creative, critical, and intelligent.

Addition Operation In numerical optimization, we have the following:

$$z = x + y \tag{19}$$

$$w = t + z \tag{20}$$

$$\frac{\partial w}{\partial x} = \frac{\partial w}{\partial z}\frac{\partial z}{\partial x} = \frac{\partial w}{\partial z} \tag{21}$$

In numerical derivatives, due to the linearity of addition, we have:

$$\frac{\partial \sum_i^N L_i}{\partial x} = \sum_i^N \frac{\partial L_i}{\partial x}. \tag{22}$$

In particular, the backward function for the addition operation passes the gradients on the output of the addition operation to its inputs.

In particular, the backward function for the addition operation passes the gradients on the output of the addition operation to its inputs.

$$\texttt{tg.sum}(L_1, \ldots, L_N) = "\{L_1\} \backslash n \{L_2\} \backslash n \ldots \{L_N\}" \tag{23}$$

Similarly, in TEXTGRAD, we have the `tg.sum` operation, that lets the gradients (feedback) on the output variable pass to the input variables.

$$\frac{\partial \texttt{tg.sum}(L_1, \ldots, L_N)}{\partial x} = \texttt{tg.sum}(\frac{\partial L_1}{\partial x}, \ldots, \frac{\partial L_N}{\partial x}) = "\{\frac{\partial L_1}{\partial x}\} \backslash n \ldots \{\frac{\partial L_N}{\partial x}\}" \tag{24}$$

where we use $+$ to indicate concatenation.

Use Case: One canonical use case for the addition operation is the batch optimization case. In particular, we implement minibatch gradient descent when performing prompt optimization (Section 3.3).

A.4 Textual Gradient Descent Implementation

Similar to backward computations, we strive to preserve generality in the implementation of TGD. In particular, we use the same glossary of tags provided above to inject information to the optimization process.

The current system prompt to the optimizer call is the following:

System prompt for the `TextualGradientDescent` optimizer.

You are part of an optimization system that improves text (i.e., variable). You will be asked to creatively and critically improve prompts, solutions to problems, code, or any other text-based variable. You will receive some feedback, and use the feedback to improve the variable. The feedback may be noisy, identify what is important and what is correct. Pay attention to the role description of the variable, and the context in which it is used. This is very important: You MUST give your response by sending the improved variable between <IMPROVED_VARIABLE> improved variable </IMPROVED_VARIABLE> tags. The text you send between the tags will directly replace the variable. {GLOSSARY}

Below is an example prompt to the optimizer:

An example prompt to the optimizer

Here is the role of the variable you will improve: <ROLE>system prompt to the language model</ROLE>.
The variable is the text within the following span: <VARIABLE> You will answer a reasoning question. Think step by step. (...) following format: 'Answer: $VALUE' where VALUE is a numerical value. </VARIABLE>
Here is the context and feedback we got for the variable:
<CONTEXT>
Here is a conversation:
<CONVERSATION><LM_SYSTEM_PROMPT> You will answer a reasoning question. Think step by step. The last line of your response should be of the following format: 'Answer: $VALUE' where VALUE is a numerical value. </LM_SYSTEM_PROMPT>
<LM_INPUT> I have a carrot, an onion, a stalk of celery, three yams, a garlic, a head of broccoli, and a potato. How many vegetables do I have? </LM_INPUT>
<LM_OUTPUT> To determine the number of vegetables you have, we need to count each individual vegetable item.
1. Carrot 2. Onion 3. Celery 4. Yams (3 yams) 5. Garlic 6. Broccoli 7. Potato
Counting all the individual vegetable items, we have a total of 7 vegetables.
Therefore, you have 7 vegetables.
Answer: 7 </LM_OUTPUT>
</CONVERSATION>
This conversation is potentially part of a larger system. The output is used as response from the language model
Here is the feedback we got for system prompt to the language model in the conversation:
<FEEDBACK>To improve the structured system prompt for the language model, consider the following feedback:
1. **Clarify the Calculation Process**: - **Current Issue**: The prompt does not explicitly instruct the model to outline each step of its calculation process. - **Improvement**: Add a directive that requires the model to explicitly state each quantity and the sum. For example, "Clearly outline each step of your calculation process, stating each quantity and the sum."
2. **Verify Intermediate Steps**: - **Current Issue**: The prompt does not instruct the model to verify each intermediate step. - **Improvement**: Include a directive for the model to verify each step against known correct values. For example, "Verify each intermediate step in your calculation to ensure accuracy."
</FEEDBACK>
</CONTEXT>
Improve the variable (system prompt to the language model) using the feedback provided in <FEEDBACK> tags. Send the improved variable in the following format:
<IMPROVED_VARIABLE>the improved variable</IMPROVED_VARIABLE>
Send ONLY the improved variable between the <IMPROVED_VARIABLE> tags, and nothing else.

B Optimizer Extensions

Batch Optimization

In batch optimization, we use the `tg.sum` function described above. In particular, gradients propagating from multiple instances are concatenated together, thus the optimizer sees all of the feedback to a variable coming from multiple sources.

The syntax is as simple as the following:

```
losses = [loss_fn(answer, model(question)) for question, answer in batch]
total_loss = tg.sum(losses)
total_loss.backward()
```

Code Snippet 4: An example use for batch optimization for question answering.

Constrained Optimization with Natural Language Constraints

In TEXTGRAD it is possible use constraints when optimizing variables. These constraints are all defined as natural language descriptions. For example, one can prompt optimizer to update the variable but to conclude its response with an answer during the update:

Example constrained optimization prompt

You must follow the following constraints:
<CONSTRAINTS>*'The last line of your response should be of the following format: 'Answer: $LETTER' where LETTER is one of ABCD."*</CONSTRAINTS>

In general, the constraint post-fix is appended to the optimizer's prompt, where the constraints are written within the `<CONSTRAINTS> {constraint text} </CONSTRAINTS>` tags.

In code, the user can simply pass in the constraints to the TGD optimizer:

```
constraints = ["The last line of your response should be of the following format: '
    Answer: \$LETTER' where LETTER is one of ABCD."]
optimizer = TextualGradientDescent(parameters=[solution], constraints=constraints)
```

Code Snippet 5: An example use for constraints when updating the solution to a problem.

Momentum

TEXTGRAD supports the use of Momentum in the Textual Gradient Descent. In standard SGD momentum uses a linear combination of past gradients and the most recent one to define a new gradient to update a variable. Similarly, TEXTGRAD keeps track of past iterations of the variable. This postfix is appended to the prompt for the optimizer.

```
optimizer = TextualGradientDescent(parameters=[solution], momentum_window=3)
```

Code Snippet 6: How to enable momentum using 3 previous steps in the `TextualGradientDescent` optimizer.

Momentum prompt

Here are the past iterations of this variable:
<PAST_ITERATIONS>{past_values}</PAST_ITERATIONS>

In-Context Examples

In-context examples can be utilized to improve textual gradients and update variables effectively. These examples serve as references to illustrate the characteristics of optimized variables. When in-context examples are applied, TEXTGRAD adopts the following prompt to incorporate them:

In-context examples prompt

You must base on the following examples when modifying the {role_description}:
<EXAMPLES>{in_context_examples}</EXAMPLES>"

By leveraging these examples, TEXTGRAD can better understand and implement the properties of optimized variables, enhancing the overall optimization process

C Code Optimization

C.1 Methodology

Baseline Details. We re-run Reflexion [26] using the code by the authors currently available online. We had to make minor changes to ensure it ran correctly in our setting but we also contacted the original authors and ask feedback on our edits to ensure our evaluation was consistent. Moreover, we had to re-extract the LeetCodeHard [26] dataset using the authors' pipeline; this means that it is likely that the dataset we are using in this manuscript is not the same dataset that was used in the Reflexion paper. While this dataset contains a set of simple tests to check if the code works as expected, the real evaluation in this context is given by passing the tests on the LeetCode platform.

Reflexion is run with a *one shot prompt* that is meant to instruct the model on how to provide feedback. The pipeline run by Reflexion is as follows: given in input a prompt to generate code, a language model generates a first solution. If this solution passes the local tests, it is then submitted to the LeetCode platform to be evaluated on harder tests. However, if the solution does not pass the tests, we ask for feedback through Reflexion and optimize the code. Once again, if the new solution passes the local tests, we submit it to the LeetCode platform. We do this optimization for 5 iterations.

We ran the experiment 5 times with 5 different seeds and we averaged the results. At each iteration of optimization, TEXTGRADmakes 1 call to `gpt-4o` to evaluate the test time loss, 1 call to collect gradients, and 1 call to update the code snippet. The number of coding problems in LeetCodeHard is 39.

Example Query for LeetCode Hard

```
def minimumTime(grid: List[List[int]]) -> int:
"""
You are given a ‘m x n‘ matrix ‘grid‘ consisting of non-negative integers where ‘grid[row][col]‘ represents the minimum time required to be able to visit the cell ‘(row, col)‘, which means you can visit the cell ‘(row, col)‘ only when the time you visit it is greater than or equal to ‘grid[row][col]‘.
You are standing in the top-left cell of the matrix in the ‘0th‘ second, and you must move to any adjacent cell in the four directions: up, down, left, and right. Each move you make takes 1 second.
Return the minimum time required in which you can visit the bottom-right cell of the matrix. If you cannot visit the bottom-right cell, then return ‘-1‘.
Example 1:
Input: grid = [[0,1,3,2],[5,1,2,5],[4,3,8,6]]
Output: 7
Explanation: One of the paths that we can take is the following:
- at t = 0, we are on the cell (0,0).
- at t = 1, we move to the cell (0,1). It is possible because grid[0][1] <= 1.
- at t = 2, we move to the cell (1,1). It is possible because grid[1][1] <= 2.
- at t = 3, we move to the cell (1,2). It is possible because grid[1][2] <= 3.
- at t = 4, we move to the cell (1,1). It is possible because grid[1][1] <= 4.
- at t = 5, we move to the cell (1,2). It is possible because grid[1][2] <= 5.
- at t = 6, we move to the cell (1,3). It is possible because grid[1][3] <= 6.
- at t = 7, we move to the cell (2,3). It is possible because grid[2][3] <= 7.
The final time is 7. It can be shown that it is the minimum time possible.
Example 2:
Input: grid = [[0,2,4],[3,2,1],[1,0,4]]
Output: -1
Explanation: There is no path from the top left to the bottom-right cell.
Constraints:
* ‘m == grid.length‘
* ‘n == grid[i].length‘
* ‘2 <= m, n <= 1000‘
* ‘4 <= m * n <= 105‘
* ‘0 <= grid[i][j] <= 105‘
* ‘grid[0][0] == 0‘
"""
```

D Solution Optimization

D.1 Methodology

For the CoT 0-shot prediction, we use the question template and system prompt released with GPT-4o in the simple-evals repository. In particular, to closely match their evaluations, we use the ChatGPT system prompt: *You are ChatGPT, a large language model trained by OpenAI, based on the GPT-4 architecture. \n Knowledge cutoff: 2023-12 \n Current date: 2024-04-01"* . Further, we use the following template:

Multiple Choice Question Answering Template

Answer the following multiple choice question. The last line of your response should be of the following format: 'Answer: $LETTER' (without quotes) where LETTER is one of ABCD. Think step by step before answering.
{Question}
A) {A}
B) {B}
C) {C}
D) {D}

During optimization, we provide the constraint to the optimizer that the prediction should conclude with an answer, following the simple-evals repository, through the following constraint description: *The last line of your response should be of the following format: 'Answer: $LETTER' (without quotes) where LETTER is one of ABCD.*.

Evaluation: Similarly, using the practice in the simple-evals repository, we perform string matching to find the final answer, which is one of the letters ABCD, and compare it to the ground truth answer. GPQA Diamond subset contains 198 questions. MMLU Machine Learning subset contains 112 questions, and College Physics subset contains 92 questions. At each iteration of optimization, we make 1 call to `gpt-4o` to evaluate the test time loss, 1 call to collect gradients, and 1 call to update the solution.

D.2 Prompts

The loss function for this task looks like the following:

Solution Refinement Objective

Below is a multi-choice question and a prediction. You are a critical and creative scientist. Your job is to investigate the prediction. Critically go through reasoning steps, and see if there is a reason why the prediction could be incorrect.
Use the Janusian Process, think about whether alternative answers could be true.
Question: {Question}
Answer by the language model: {**Solution**}

Example Query for GPQA Diamond

What is the concentration of calcium ions in a solution containing 0.02 M stochiometric Ca-EDTA complex (we assume that the pH is ideal, T = 25 °C). KCa-EDTA = $5x10^10$.
A) $5.0x10^{-}3$ M
B) $1.0x10^{-}2$ M
C) $6.3x10^{-}7$ M
D) $2.0x10^{-}2$ M

E Prompt Optimization

E.1 Tasks

Below, we provide an example query for each of the tasks in the prompt optimization section.

Example Query for Word Sorting

Sort the following words alphabetically: List: oakland seaborg jacobi membrane trapezoidal allis marmot toggle anthology.

Example Query for Object Counting

I have a couch, a bed, a car, a fridge, two tables, an oven, a toaster, and a chair. How many objects do I have?

Example Query for GSM8k

Amber, Micah, and Ahito ran 52 miles in total. Amber ran 8 miles. Micah ran 3.5 times what Amber ran. How many miles did Ahito run?

For word sorting and object counting, we obtain the datasets from the BBH repository, and we randomly split examples into 50 (training)/100 (validation)/100 (test) samples. For GSM8k, we use the splits provided in DSPy [10] which has 200 (training)/300 (validation)/1319 (test) samples.

Evaluation: For object counting and GSM8k, we use the string-based exact match metric, which looks at the last numerical value provided in the answer, and compares it to the ground truth answer. For word sorting, we prompt `gpt-4o` to compare the ground truth list to the response provided in the answer, through the following prompt:

Evaluation system prompt for Word Sorting evaluation

System Prompt: Below is a question from a question-answering task, the ground truth answer, and reasoning with the final prediction. Is the final prediction correct, i.e. the same as the ground truth answer? Say only 1 (yes) or 0 (no). Return your response within <ACCURACY> </ACCURACY> tags. e.g.<ACCURACY> 0 </ACCURACY> or <ACCURACY> 1 </ACCURACY>.

Example prompt:
Question for the task: {question}
Ground truth answer: {answer}
Reasoning and prediction from the language model: {prediction}

F Molecule Optimization

F.1 Docking and Druglikeness Evaluation

To optimize molecules, we evaluate the binding affinity and druglikeness of chemical structures encoded as SMILES strings. To compute both metrics, the generated SMILES string is first converted into an octet-complete Lewis dot structure using RDKit's `MolFromSmiles` functionality. This method "sanitizes" molecules by adding explicit hydrogens, kekulizing aromatic rings, standardizing valence states, and assigning radicals [100]. If this sanitization process fails at any step, whether through a structural ambiguity or an invalid molecule, the QED and Vina scores are replaced with a single string informing TEXTGRAD that the molecule is invalid.

If the generated SMILES string does represent a valid chemical structure, we compute the QED score using RDKit's `Chem.QED` function. While QED scores can be quickly and reliably computed, docking scores can vary significantly depending on target and ligand structure preparation. To ensure consistency, we calculate Vina scores using the DOCKSTRING package [61], which implements a standardized docking workflow and provides a set of gold-standard targets. In particular, after the ligand has been sanitized by RDKit, DOCKSTRING (de)-protonates it at pH 7.4 using Open Babel, and prepares and refines the 3D geometric structure using the Euclidean distance geometry algorithm ETKG and the classical force field MMFF94. Finally, Gasteiger charges are computed for all ligand atoms, and the resulting structure is saved as a ligand PDBQT file passed to Autodock Vina. For target preparation, the DOCKSTRING benchmark suite provides 58 curated crystal structures of clinically relevant proteins, with the majority at less than 2.5 Å resolution. These structures are specially prepared to improve correlations between theoretical and experimental binding affinities, for example by manual addition of polar hydrogens and removal of residual water and solute molecules. DOCKSTRING also standardizes simulation parameters such as numerical seeds, search box coordinates, and sampling exhausitivity, to ensure reproducible scoring.

F.2 Objective Functions

Once the scores have been computed, they are passed to an LLM in the following format.

Molecule Optimization Prompt

Given a docking and a druglikeness score, and a molecule as a SMILES string provide a short criticism to improve the druglikeness of this molecule and its binding affinity with the protein {`protein_name`}. For docking, lower is better (less than -10 is considered good) and for druglikeness, 1 is the best and 0 is the worst (greater than 0.8 is considered good). In terms of prioritization, the docking score is {`vina_qed_ratio`} times as important as the druglikeness score. Make sure your criticism is very succinct and to the point.

`# if smiles_string is valid`
SMILES: {`smiles_string`}, Docking: {`Vina`}, Druglikeness: {`QED`}

`# if smiles_string is invalid`
SMILES: {`smiles_string`}, This molecule is invalid.

Note that this prompt allows us to specify both the *target name* as well as a *prioritization* between these two objectives. When optimizing promising molecules in late stage drug discovery, medicinal chemists use detailed structural knowledge about a protein target's binding pocket in addition to docking scores, for example through interactive 3D molecular visualization software. To preserve the generality of TEXTGRAD in our experiments, we do not to attempt to provide similar geometric information, as not all LLMs support multimodal inputs. However, we do include the `protein_name` in the loss prompt to inject supplementary structural information.

In practice, drug efficacy is typically a higher priority than absorbtion efficiency [101, 102], so we set the `vina_qed_ratio` to be 10. Empirically, we observe that by scaling this prioritization factor, we can tune TEXTGRAD's generation towards molecules with differing binding affinity and druglikness tradeoffs. This prioritization also allows us to simplify post-hoc selection and ranking of the generated molecules with a single "overall" score defined as follows, where a lower overall score $s_{\texttt{overall}}$ indicates a better molecule.

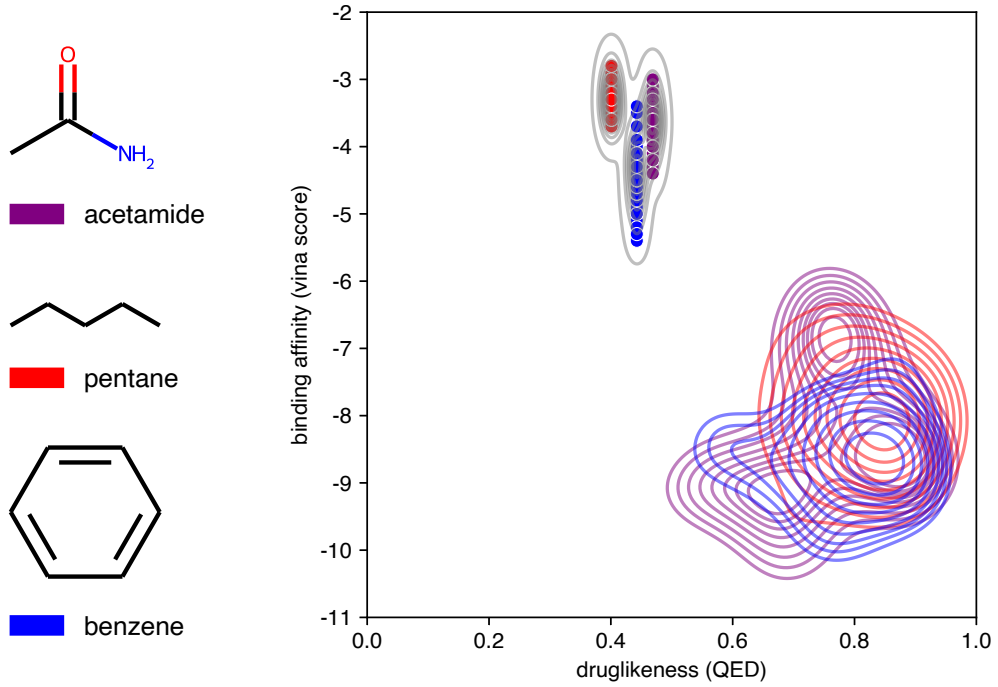
$$s_{\texttt{overall}}(\text{molecule}, \text{protein}) = \texttt{Vina}(\text{molecule}, \text{protein}) + (1 - \texttt{QED}(\text{molecule})) \tag{25}$$

Since the QED score is bounded between 0 and 1, and the Vina score typically ranges between -3.0 to -12.0 kcal/mol, this overall score places approximately 10 times more emphasis on binding affinity than druglikeness.

F.3 Benchmarks

To benchmark the performance of TEXTGRAD generated molecules, we compare their characteristics to clinically approved drugs for the same protein targets found in DrugBank, a database of 16,619 drugs [103]. To ensure that the DrugBank molecules were both comparable and high quality, we filtered for drugs that were small molecules, had full clinical approval, and were designed for orthosteric binding with the same active site in the DOCKSTRING benchmark suite. After these filtering criteria, we identified 118 drugs targeting 29 of the 58 DOCKSTRING proteins. When evaluating binding affinity and druglikeness for the clinically approved drugs, we compute the Vina and QED scores using the same tools and workflow described in section F.1, exactly as applied to the TEXTGRAD molecules.

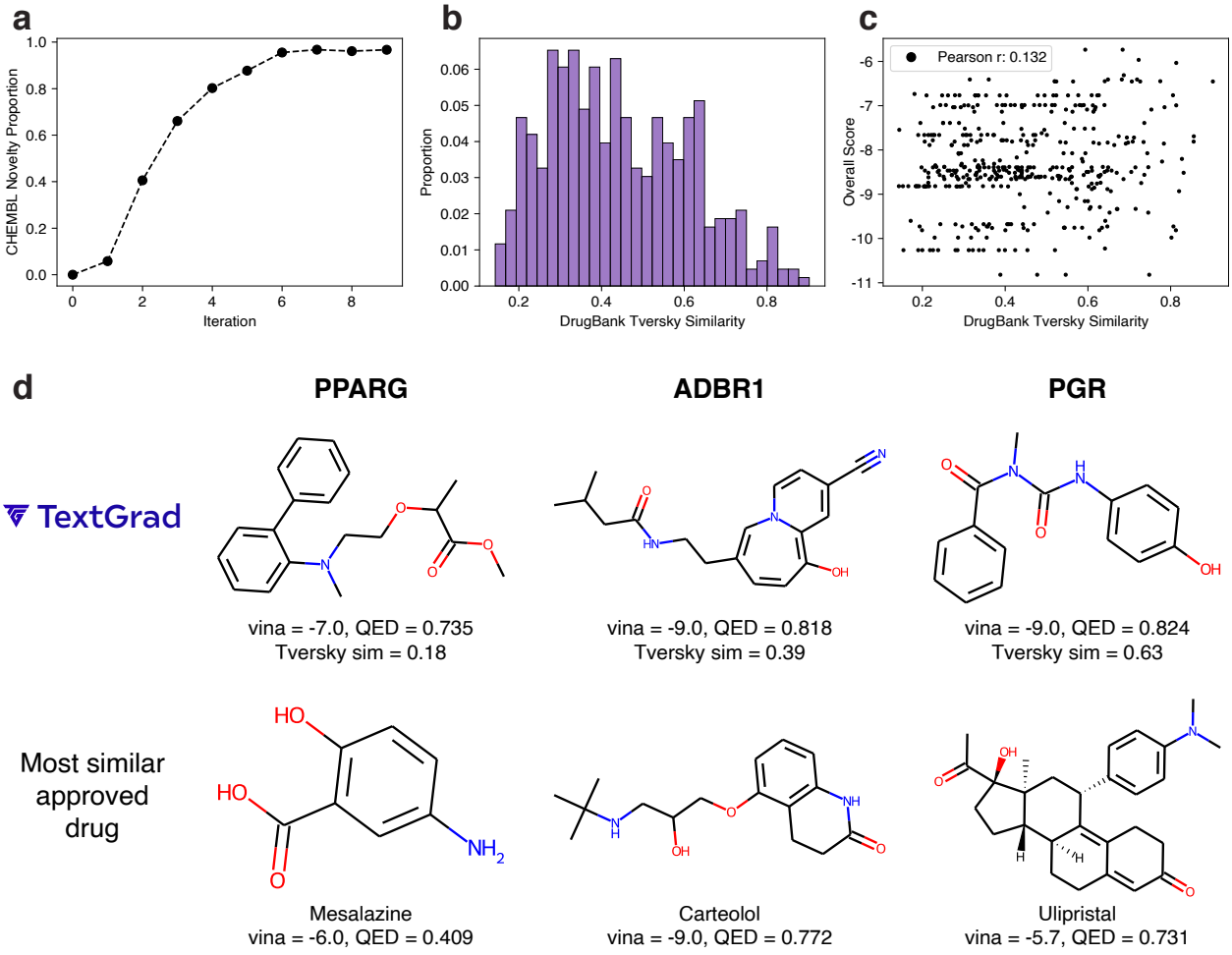
F.4 Initialization



Supplementary Figure 1: Molecule Initialization. We initialize TEXTGRAD with fragment molecules from three, diverse functional groups and optimize each initial molecule for 10 iterations for all 58 targets in the DOCKSTRING benchmark suite. For each fragment and each protein, we perform post selection on the generated molecules using the summary score outlined in equation 25 and visualize the resulting distribution. We observe that while the initial QED and Vina score distributions of the starting fragments varies greatly, the distribution of the optimized molecules is highly overlapping.

In practice, molecular optimization is typically accelerated by large scale pre-optimization screening, where libraries of millions or even billions of existing chemical structures are scored and ranked by docking, druglikeness, and other metrics. Only the most promising structures, termed "leads", are further refined by medicinal chemists [104]. While TEXTGRAD is capable of optimizing any initial molecule, in this work, to more accurately characterize it's performance and avoid biasing its designs towards existing drugs, we instead select our three initial molecules from simple fragments of common functional groups, shown in Figure 1. These fragments are highly diverse, and belong to different functional groups. Although these initial fragments have differing druglikness and binding affinity characteristics, the impact of the starting fragment on TEXTGRAD's performance is minimal, and TEXTGRAD is capable of generating molecules with high druglikeness and binding affinity from all three fragments.

F.5 Chemical Novelty



Supplementary Figure 2: Structural Novelty In panel **(a)**, we observe increasing novelty over optimization updates, where a generated molecule is considered"novel" if there does not exist any existing molecule in ChEMBL with a Tanimoto similarity score greater than 0.8. In panels **(b,c,d)**, we estimate substructure similarity using the Tversky metric between TEXTGRAD molecules and clinically approved drugs for each target. While there are a wide range of similarity scores, we observe that similarity with known drugs has little correlation with molecule performance, suggesting that TEXTGRAD's generation process is weakly influenced by "memorized" knowledge of clinically approved structures.

One of the primary concerns with using LLM models in scientific discovery is their capacity to simply memorize and regurgitate their training sets instead of performing logical reasoning [105]. Since recent models like `gpt-4o` are trained on massive, opaque datasets, a key concern is that the molecules TEXTGRAD generates may simply be duplicates of clinically approved molecules for their respective targets. To quantify and test the extent or existence of this "memorization" effect, we compare the molecules that TEXTGRAD generates with approved drugs for their respective protein targets, as well as unrelated but existing druglike chemical compounds.

We first compare TEXTGRAD generated molecules for a particular protein target with all the clinically approved small molecules found in Drugbank for the same target (see F.3 for filtering criteria) using the Tversky similarity score on RDKit daylight chemical fingerprints. The Tversky similarity score is not symmetric, and compares substructures between a variant and reference molecule [106]. In our application, this is preferred as it assigns a high score to generated (variant) molecules that contain most or all of the sub-

structures in the Drugbank (reference) molecule, even if there exist other extraneous substructures in the generated molecule that reduce the symmetric overlap between the two molecules. A Tversky score of 1.0 indicates that the TEXTGRAD molecule is a complete subset of the Drugbank molecule, while 0.0 indicates no overlap.

In order to analyze the most relevant chemical structures, we restricted our analysis to a set of high performing TEXTGRAD molecules, rather than all molecules from all iterations. In particular, using the overall score described in Equation 25, we selected the best molecule for each protein target and for each initial fragment, for a total of 87 generated molecules (29 druggable targets x 3 initial fragments) and 118 clinically approved drugs across 29 targets. For each target, we compute all pairwise Tversky scores between the 3 TEXTGRAD molecules and the Drugbank molecules approved for that target, setting the Drugbank molecule as the reference, and the TEXTGRAD molecule as the variant.

We observed that the distribution of Tversky similarity scores was quite broad, with a median of 0.42 but ranging from 0.14 to 0.90 (Figure 2 **(b)**). However, we observe that Tversky similarity between TEXTGRAD and DrugBnak molecules is actually slightly anti-correlated with molecule performance as measured by the overall score, suggesting that TEXTGRAD 's optimization procedure is at best weakly influenced by prior knowledge of approved drugs. In fact, for a variety of generated molecules along a gradient of similarity scores, the generated molecules exhibit QED and Vina scores that match or even exceed their DrugBank counterparts (Figure 2 **(d)**).

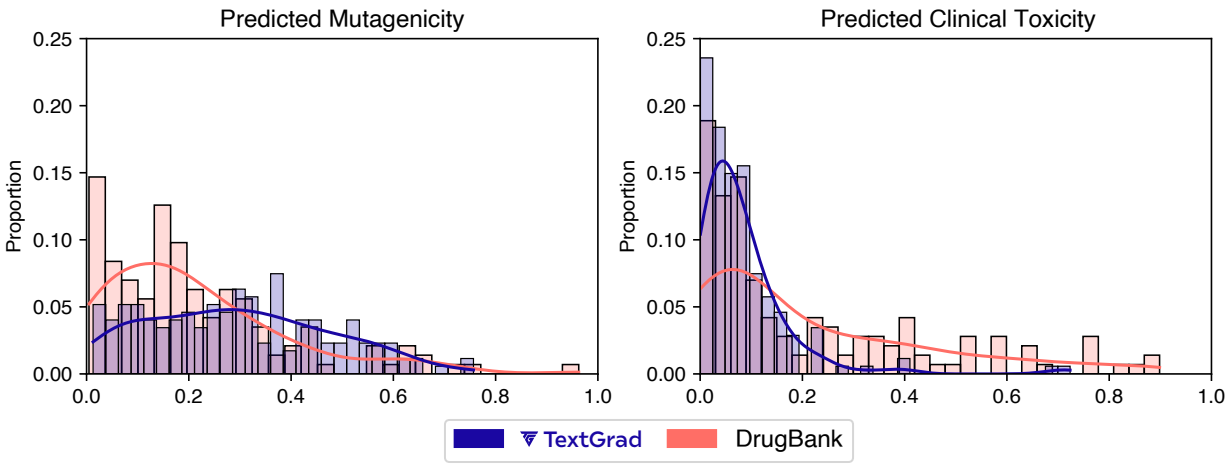
Beyond the similarity to known drugs, we are also interested in observing if TEXTGRAD is generating novel molecules, or discovering previously unknown properties in existing compounds, for example a strong binding affinity to a protein target in a compound that has not been associated with the protein. To answer this question, we perform a Tanimoto similarity search across all of ChEMBL, a manually curated database of 2.4 million bioactive molecules with drug-like properties [107]. Unlike the Tversky score, the Tanimoto metric measures the symetric overlap between two chemical structures, where 1.0 indicates an exact match, and 0.0 no overlap. We classify a TEXTGRAD compound as "novel" if and only if there does not exist any molecule in ChEMBL with a with a Tanimoto similarity score over 0.80.

While the starting fragments are known molecules, we observe that as the number of iterations increases, TEXTGRAD generates molecules that are progressively less likely to be previously known compounds. By the 6th iteration, 95% of all the molecules generated by TEXTGRAD across all 58 targets and 3 starting fragments are novel using the criteria above (Figure 2 **(a)**). By observing the trajectories of generated molecules and analyzing the textual gradients, we hypothesize that the feedback provided by the Vina and QED scores encourages TEXTGRAD to explore chemical space beyond known molecules by progressively adding and removing combinations of functional groups. Since these chemical updates can form a combinatorial number of unique structures, it is reasonable that TEXTGRAD would reach previously unexplored regions of chemical space in a relatively small number of iterations.

F.6 Implicit Objectives

Another key concern in applying LLMs in science is their propensity for hallucinations, where models generate factually incorrect or illogical responses while attempting to satisfy user requests [105]. In our setting, this hallucinations could manifest by TEXTGRAD proposing invalid, toxic, or otherwise undesirable molecules in order to optimize its objective function. We control for severe hallucinations by preprocessing molecules using RDKit sanitization, ensuring at the bare minimum that TEXTGRAD generates chemically valid molecules. However, this simple preprocessing step does not completely specify desirable chemical behavior. A direct strategy would be to exhaustively encode all possible metrics for desirability in drug molecules beyond druglikeness and binding affinity into the objective function, and extend it to include sythesizability, toxicity, among other criteria. Unfortunately, this approach is not realistically feasible as not all criteria for desirability have mature computational metrics. Thus, a key question is whether TEXTGRAD obeys so-called *implicit* objectives during its optimization process that curtails illogical or undesirable behavior.

To evaluate the extent or existence of undesirable molecules, we can characterize the harmfulness of the generated molecules, focusing on mutagenisis and clinical toxicity. Mutagenicity refers to the ability of a drug to induce genetic alterations, which may lead to DNA damage and harmful long-term affects. Clinical toxicity refers to a broad range of adverse short or long-term side effects. Importantly, neither druglikeness



Supplementary Figure 3: Safety Properties We evaluate the predicted harmfullness of TEXTGRAD molecules using the ADMET-AI model and compare them to clinically approved drugs. For both mutagenicity and clinical toxicity, 1.0 indicates a highly likelihood for harm and 0.0 a low likelihood. We observe that despite the fact that neither of these characteristics are directly encoded into TEXTGRAD's objective function, TEXTGRAD implicitly avoids proposing harmful molecules.

nor binding affinity are strongly correlated with these criteria, and thus these desirability metrics are not directly optimized or otherwise explicitly encoded in TEXTGRAD's objectives.

To evaluate the propensity for mutagenisis and clinical toxicity, we employ the ADMET-AI model, that predicts these scores from the chemical structures of molecules [108]. ADMET-AI employs a deep learning model trained on multiple relevant datasets. In particular, for mutagenicity, ADMET-AI is trained on 7,255 drugs from the Ames dataset, a bacterial reverse mutation assay for rapidly screening large numbers of compounds for can induced genetic damage and frameshift mutations. A predicted label of 1.0 indicates a high likelihood that the drug will induce mutagenesis, while a label of 0.0 indicates a low likelihood. For clinical toxicity, ADMET-AI is trained on the ClinTox dataset, a dataset of 1,484 drugs consisting of molecules that have failed clinical trials for toxicity reasons and also drugs that are associated with successful trials. Similarly, a predicted label of 1.0 indicates a high likelihood of clinical toxicity, while a label of 0.0 indicates a low likelihood.

Once again, we restrict our analysis to the best performing generated molecules with druggable targets as measured by the overall score, and select the best molecule for each protein target and for each initial fragment, for a total of 87 generated molecules. We then compare their predicted mutagenisis and clinical toxicity to the the 118 clinically approved molecules from DrugBank. We observe that for both Mutagenicity and Clinical Toxicity, the molecules generated by TEXTGRAD have predicted distributions that indicate a low likelihood of harmful effects, and closely match the distributions of the clinically approved molecules. Together, these results suggest that TEXTGRAD implicitly avoids proposing harmful molecules, even though these criteria are not directly encoded in its loss function.

G Treatment Plan Optimization

G.1 Prompts

Radiotherapy treatment plan evaluation can based on various dimensions, therefore there is no single score that can indicate the quality of plans. We adopt LLM to compute the "loss" by prompting it to assess the plan quality with clinical protocols. Specifically, LLM is used to compare each protocol with the current plan and produce the final assessment.

Treatment Plan Loss Prompt

Please act as an impartial and objective professional radiation oncologist. Your job is to evaluate the quality of radiation therapy treatment plans based on their dose volume histogram. You should consider the following protocols:
<Clinical Protocols>
Note that the order above does not indicate priority; always prioritize the regions that have protocols that are more significantly violated.
Here is the dose-volume histograms of the candidate plans for evaluation; each entry in the dose-volume histograms (DVH) table indicates the percentage of volume receiving a dose higher than a certain Gy (specified in the first column).
<DVH table>
We also provide the statistics of the above DVH table for ease of evaluation.
<DVH statistics>
Now, based on the protocols, and the DVH, please evaluate the plans. Avoid any positional biases and ensure that the order in which the responses are presented does not influence your decision. Be as objective as possible. Your answer must:

1. (Evidence) Extract the corresponding entries from the DVH table based on the factors to consider.
2. (Interpretation) Based on the extracted entries from DVH table, interpret them as: `{percentage}`% of volume encompassing the dose more than `{dose}` Gy for `{organ}`.
3. (Analysis) Combine the interpretation and the factors, analyze whether each of the numerical factors are met.

Based on the analysis on DVH tables, you need to produce a final evaluation. When all factors are satisfied, you should always answer no improvement is needed. If improvements are required, please provide suggestions on where to improve. Please ensure to follow the format below to return the evaluation results:

<FINAL> Decision: [The plan does/doesn't need to be improved.] Reasons: [list all factors that are not satisfied with detailed reasons, e.g. protocol X on (PTV/rectum/bladder/fh/body) is not satisfied because ...] </FINAL>

The final answer in the end must strictly follow the format above.

G.2 Inner-loop optimization for treatment planning

We employ a two-loop optimization approach [109], which includes (i) an inner loop for inverse planning and (ii) an outer loop for optimizing the hyperparameters of the inner loop. The inner loop focuses on traditional fluence map optimization, seeking to determine the optimal fluence map x by minimizing a cost function that combines multiple weighted objectives for various targets and organs at risk. This cost function is defined as:

$$\begin{aligned} \min_{x} \quad & \sum_{t=1}^{N_t} w_t([Kx]_t - d_t)^2 + \sum_{s=1}^{N_s} w_s \Theta([Kx]_s - d_s)([Kx]_s - d_s) \\ \text{s.t.} \quad & D_{95}([Kx]_t) = d_t, \\ & x \geq 0, \end{aligned} \tag{26}$$

Here, $\{w_t\}_{t=1}^{N_t}$ and $\{w_s\}_{s=1}^{N_s}$ are the importance weights (the hyperparameters optimized by TEXTGRAD) that balance the various objectives for N_t PTV targets and N_s OARs, respectively. K denotes the dose

influence matrix, which specifies the dose per fluence unit delivered to each voxel in the volume by each beamlet. $\{d_t\}_{t=1}^{N_t}$ and $\{d_s\}_{s=1}^{N_s}$ are the scalar objective doses for each structure. The cost function essentially penalizes squared deviations from the target objective doses for the PTV targets and penalizes squared overdosing for the OARs only when doses exceed d_s. The Heaviside function Θ is used to ensure the objective considers only positive values. The minimization is constrained to positive x values and ensures that D_{95} — the minimum dose received by 95% of the structure volume — matches the prescribed dose for the clinical goal. We use `matRad` [65] with interior point algorithms to solve the inner-loop optimization.

G.3 Additional Experimental Details

Dataset The dataset used in this study comprised imaging and treatment plans for 5 prostate cancer patients who underwent intensity-modulated radiation therapy (IMRT). Available data for each patient includes CT scans, delineated anatomical structures, and clinically approved treatment plans obtained via Eclipse®.

Method As we mentioned in 3.5, TEXTGRAD is used to optimize the hyperparameters (e.g., importance weights for PTV and OARs) of the inner-loop numerical optimizer that generates the treatment plan. This optimization is done using a variation of vanilla TEXTGRAD, i.e. "projected gradient descent with momentum updates".In particular, three prostate cancer treatment plans optimized by clinicians, along with their corresponding hyperparameters, are provided. These examples guide the updates of the hyperparameters. This procedure can be viewed as an analogy to projection, as the updated hyperparameters are "softly projected" onto a feasible set defined by the three in-context examples. Moreover, the historical hyperparameters and the textual gradients from past iterations, as an analogy to momentum, are also included in the prompts for updating the hyperparameters. This additional context helps refine the optimization process. The optimization will be stopped if the loss suggests all protocols meet, other wise, it will be stopped if the maximum number of iterations (we set it to 10) is reached.

Initialization The hyperparameters i.e. the importance weights are all initialized at 100 for different organs. The dose objectives are set to 70.20 for PTV, 0.00 for bladder and rectum, and 30.00 for femoral heads and body, and fixed during optimization.

G.4 Additional Results

In Supplementary Table 1 and 2, we show additional results on comparing TEXTGRAD optimized plan with clinicians optimized plans.

Supplementary Table 1: PTV dose metrics. Several dose metrics of the PTV target are displayed for all the clinical and TextGrad optimized plans, including the mean and minimum doses, as well as the D_{95}. For all the metrics, we include the average deviations from the clinical goal across 5 plans and the standard deviation in brackets. Values in bold represent the best for each PTV target.

Target	Method	Mean dose [Gy]	Min dose [Gy]	Max dose [Gy]	D_{95} [Gy]
PTV	Clinical Goal	70.20	$\approx$ 70.20	$\approx$ 70.20	70.20
	Radiation Oncologist	+1.97 (0.36)	-8.88 (2.31)	+4.66 (0.82)	-0.10 (0.15)
	TEXTGRAD	**+0.51** (0.09)	**-8.48** (2.38)	**+3.63** (0.87)	**+0.00** (0.00)

Supplementary Table 2: Organs at Risk (OARs) dose metrics. We show mean dose capturing OAR sparing. Lower values demonstrate better OAR sparing which is desirable, as this number indicates organs at risk, which should not get more than dosage than what is listed in the clinical guidelines. For all the metrics, we include the average mean dose across 5 plans and the standard deviation in brackets.

Organ	Method	Mean dose [Gy] ↓	D_5↓	D_{50}↓
Rectum	Radiation Oncologist	23.88 (6.45)	64.26 (10.00)	20.04 (5.50)
	TEXTGRAD	**17.18** (4.2)	**58.82** (18.81)	**9.54** (0.70)
Bladder	Radiation Oncologist	22.39 (5.55)	67.81 (6.44)	14.78 (8.42)
	TEXTGRAD	**20.92** (0.79)	**65.96** (6.96)	**14.11** (3.17)